

Fundamentals of Computers and Computational Thinking

Course Code: CS101 Credits: 4 (Lecture: 3, Practical: 2) Total Hours: 75 Hrs Total Evaluation: 100 Marks (ESE: 70, CIA: 30)

Comprehensive Course Table of Contents

Module I: History and Evolution of Computers

- Unit 1:** Introduction to Computers — Definition & Architecture
- Unit 2:** Historical Evolution — Abacus to Modern Era
- Unit 3:** Computer Generations (1st to 5th)
- Unit 4:** Classification — Scale & Operating Principles
- Unit 5:** Computing Paradigms — Embedded to Quantum

Module II: Hardware and Components

- Unit 6:** Electronic Components — Active & Passive
- Unit 7:** Motherboard Architecture & System Buses
- Unit 8:** Power Supply — SMPS & Power Regulation
- Unit 9:** Primary Storage — RAM & ROM Technologies
- Unit 10:** Secondary Storage — Magnetic, Optical, Solid-State
- Unit 11:** Input/Output Interfaces & Display Subsystems

Module III: Software and Operating Systems

- Unit 12:** Software Classification — System vs Application
- Unit 13:** Operating System Fundamentals & Boot Sequence
- Unit 14:** File Systems & Partitioning — MBR vs GPT
- Unit 15:** Memory & Peripheral Management — Interrupts & DMA

Module IV: Computational Thinking, Logic & Algorithms

- Unit 16:** Computer Science as a Discipline — Scope & Challenges
- Unit 17:** Problem Solving Fundamentals & SDLC
- Unit 18:** Four Pillars of Computational Thinking
- Unit 19:** Logical Reasoning & Propositional Logic
- Unit 20:** Algorithm Design & Complexity Analysis
- Unit 21:** Flowcharts & Visual Modeling (ISO 5807 / RAPTOR)
- Unit 22:** Practical Lab Implementations (Problems 1 to 6)

Fundamentals of Computers and Computational Thinking

Course Code: CS101 **Credits:** 4 (Lecture: 3, Practical: 2) **Total Hours:** 75 Hrs

MODULE I: History and Evolution of Computing System

Comprehensive Academic Notes — Units 1 through 5 (Duration: 9 Hours, Weightage: 15 Marks)

Unit 1: Evolution of Computers — History & Generations

CONCEPTUAL DEFINITION: THE MODERN COMPUTER

A **computer** is a multi-purpose, programmable, electronic data-processing machine engineered to automatically accept raw, unstructured data as input, apply high-speed mathematical, arithmetic, and logical operations in accordance with a sequence of internally stored instructions, produce structured, meaningful output information, and persist intermediate and final results in non-volatile storage media for future retrieval and processing.

1.1 The Pre-Electronic and Mechanical Computing Eras

The intellectual and technological evolution of computation spans several millennia. Human civilization has continuously sought to automate tedious mathematical calculations, reduce human error in numerical tabulation, and expand the horizons of abstract problem-solving.

The earliest known calculation aid is the **Abacus**, dating back to circa 2500 BCE in ancient Mesopotamia and subsequently perfected in China as the *Suanpan* and in Japan as the *Soroban*. The abacus utilizes a rectangular wooden frame with parallel vertical rods upon which sliding beads represent positional numerical quantities (units, tens, hundreds, thousands). Skilled operators could execute addition, subtraction, multiplication, division, and even square-root extractions at speeds comparable to early mechanical adding machines.

In 1617, Scottish mathematician John Napier published his treatise *Rabdologiae*, presenting **Napier's Bones**. This invention consisted of a set of carved ivory, wood, or bone rods inscribed with multiplication tables. Utilizing the principle of lattice multiplication (the gelosia method), Napier's bones converted complex multi-digit multiplication and division into a series of elementary single-digit additions and subtractions. Furthermore, Napier's contemporaneous discovery of logarithms enabled the creation of the **Slide Rule** (developed by William Oughtred in 1622), an analog computational tool featuring sliding logarithmic scales that remained the premier calculation instrument for scientists and engineers through the mid-20th century.

The transition from manual calculation aids to mechanical gear-driven machinery began in 1642 with the French mathematician, physicist, and philosopher Blaise Pascal. To assist his father, a regional tax administrator in Rouen, the 19-year-old Pascal designed and constructed the **Pascaline** (or *Machine Arithmetique*). The Pascaline was the first operational mechanical calculator based on a series of interlinked ten-toothed wheels representing decimal digits (**0** through **9**). A sophisticated internal gravity-assisted mechanical pawl system (the *sautoir*) executed an automatic "carry" operation whenever a wheel completed a full **360°** rotation from digit **9** to **0**. While the Pascaline could only perform addition directly and subtraction through nines' complement addition, it proved that mathematical logic could be physically embodied in mechanical clockwork.

In 1673, German polymath Gottfried Wilhelm von Leibniz dramatically expanded Pascal's concepts by inventing the **Stepped Reckoner** (*Staffelwalze*). Leibniz introduced the stepped gear cylinder (a cylindrical drum with nine teeth of incrementally increasing lengths), which allowed a movable carriage to execute direct, automated multiplication and division through repeated stepped rotations, alongside addition and subtraction. Remarkably, Leibniz was also the first Western mathematician to comprehensively document the mathematical foundations of the **Binary Number System** (using only **0** and **1**), recognizing its profound simplicity and suitability for mechanical and philosophical computation.

In 1804, a pivotal conceptual breakthrough occurred in textile manufacturing that would fundamentally alter computing history: Joseph Marie Jacquard invented the **Jacquard Loom**. Jacquard automated the weaving of intricate pictorial tapestries and damask patterns by reading sequences of rigid punched pasteboard cards. Holes punched in specific positions allowed mechanical needles to pass through and lift corresponding warp threads, while unpunched card areas blocked the needles. The Jacquard Loom introduced two revolutionary concepts: **binary-encoded data storage** (hole = 1, no hole = 0) and the principle of **program-controlled sequential machine execution** without altering physical mechanical gearing.

1.2 The Five Generations of Electronic Computing

The progression of electronic digital computers is historically categorized into five distinct generations. Each generational epoch is characterized by a revolutionary paradigm shift in active switching electronics, primary memory media, computational throughput, operating system capabilities, and programming language abstractions.

First Generation (1940–1956): The Era of Vacuum Tubes (Thermionic Valves)

The first generation of electronic digital computers relied on **vacuum tubes** (thermionic valves invented by John Ambrose Fleming and Lee de Forest) as the fundamental active switching and amplifying element. A vacuum tube consists of an evacuated glass or metal-ceramic envelope enclosing a heated filament cathode, a control grid, and an anode plate. By applying an electrical charge to the grid, the flow of thermionically emitted electrons from cathode to anode could be modulated or switched ON and OFF at kilohertz frequencies, far exceeding mechanical relays.

- **Physical & Electrical Attributes:** First-generation systems were colossal, room-sized mainframe installations weighing between **10** and **30** metric tons and occupying thousands of square feet of floor space. They consumed massive quantities of electrical power (frequently exceeding **150 kW**) and generated tremendous heat, requiring dedicated industrial air conditioning. Because vacuum tube tungsten filaments operated at incandescent temperatures, they suffered from extremely high failure rates (Mean Time Between Failures — MTBF — was often measured in minutes or hours), requiring teams of technicians to manually diagnose and replace burnt-out tubes.

- **Memory Subsystem:** Primary memory technologies were primitive and severely capacity-constrained. Early systems employed **mercury acoustic delay lines** (where electrical pulses were converted into acoustic sound waves traveling through long tubes of liquid mercury, reflected, and recirculated), **Williams-Kilburn electrostatic tubes** (using cathode ray tube phosphor screens to store static charges), and high-speed rotating **magnetic drums** (spinning at **3,000 RPM** where magnetic tracks stored binary data). Memory capacities ranged from 1,000 to 4,000 words.
- **Software & Programming:** These machines operated without operating systems. All software was written directly in **Machine Language** (raw binary strings of **0s** and **1s**) or hardwired by manually configuring physical plugboards, patch cords, and rotary switches. Input and output were handled via 80-column punched cards and magnetic tape reels.
- **Seminal First-Generation Computers:**
 - **ENIAC (Electronic Numerical Integrator and Computer):** Completed in 1945 by John Mauchly and J. Presper Eckert at the University of Pennsylvania's Moore School of Electrical Engineering. Containing **17,468** vacuum tubes, **70,000** resistors, **10,000** capacitors, and **5 million** hand-soldered joints, ENIAC executed **5,000** additions per second — approximately **1,000** times faster than any electromechanical calculator of its day.
 - **EDVAC (Electronic Discrete Variable Automatic Computer):** Conceived in 1945 as the first binary, stored-program electronic computer.
 - **UNIVAC I (Universal Automatic Computer I):** Delivered to the United States Census Bureau in 1951, UNIVAC I was the world's first commercially mass-manufactured general-purpose electronic computer. It achieved worldwide acclaim when it accurately predicted the landslide victory of Dwight D. Eisenhower in the 1952 US Presidential Election on live national television.

Second Generation (1956–1963): The Transistor Revolution

The invention of the point-contact transistor at Bell Laboratories in December 1947 by physicists John Bardeen, Walter Brattain, and William Shockley (who shared the 1956 Nobel Prize in Physics) catalyzed the second generation of computing. The transistor is a three-terminal solid-state semiconductor device that controls current flow through solid silicon or germanium crystals without requiring heated filaments or a vacuum.

- **Technological Advantages:** Discrete transistors reduced physical component volume by a factor of **1,000**, slashed electrical power consumption by over **95%**, generated minimal heat, operated at megahertz switching frequencies, and eliminated filament burnout, boosting machine reliability by multiple orders of magnitude. Computational execution times dropped from milliseconds (**10^{-3} s**) to microseconds (**10^{-6} s**).
- **Memory Revolution — Magnetic Core Memory:** Second-generation systems replaced acoustic delay lines with **Magnetic Core Memory**. Developed by Jay Forrester and An Wang, core memory comprised tiny toroidal ferrite rings (cores) threaded onto a two-dimensional orthogonal wire grid (X and Y drive lines, plus sense and inhibit wires). By passing half-select currents along intersecting wires, individual cores were magnetized clockwise (representing binary **1**) or counter-clockwise (representing binary **0**). Core memory provided non-volatile, random-access memory (RAM) with access cycle times of **1 to 2 microseconds**.
- **Software Advancements:** The second generation saw the birth of **Assembly Languages** (which replaced machine code with mnemonic abbreviations such as **MOV**, **ADD**, **SUB**, and symbolic memory addresses) and the emergence of the first standardized **High-Level Programming Languages (HLL)**:
 - **FORTRAN (Formula Translation):** Developed in 1957 by John Backus at IBM for scientific and engineering calculation.
 - **COBOL (Common Business-Oriented Language):** Formulated in 1959 by the CODASYL committee, heavily guided by Rear Admiral Grace Hopper, designed for human-readable business data processing.
 - **ALGOL (Algorithmic Language):** Introduced in 1958/1960, establishing block structures, lexical scoping, and formal grammar syntax (Backus-Naur Form - BNF).

Primitive resident monitor programs emerged, serving as early ancestors of batch-processing operating systems.

- **Representative Systems:** IBM 7090, IBM 1401 (the most popular commercial computer of its era, with over 12,000 units leased), CDC 1604, and UNIVAC 1107.

Third Generation (1964–1971): Integrated Circuits (ICs) & Time-Sharing

The third generation began with the commercial deployment of the **Integrated Circuit (IC)**, independently invented by Jack Kilby of Texas Instruments (who created a hybrid germanium circuit in 1958) and Robert Noyce of Fairchild Semiconductor (who patented the monolithic silicon planar IC in 1959).

- **Monolithic Integration:** Instead of soldering thousands of discrete transistors, diodes, and resistors onto circuit boards, the planar process allowed entire circuits containing dozens to hundreds of electronic components to be microscopically etched and diffused onto a single monolithic wafer of monocrystalline silicon. This era spanned **Small Scale Integration (SSI, < 10 logic gates per chip)** and **Medium Scale Integration (MSI, 10 to 100 gates per chip)**.
- **Performance and Storage:** Computational instruction cycle speeds reached nanoseconds (**10^{-9} s**). Semiconductor solid-state RAM chips (such as the Intel 1103 DRAM chip introduced in 1970) began displacing magnetic core memory. Mass-storage magnetic disk packs with movable read/write heads provided tens of megabytes of online, random-access storage.
- **Operating Systems & Multiprogramming:** The third generation revolutionized software architecture. Instead of processing single batch jobs sequentially, sophisticated **Multiprogramming and Time-Sharing Operating Systems** (such as IBM OS/360, MIT's CTSS, and MULTICS) were developed. Multiple concurrent users could interact with a central mainframe simultaneously via remote alphanumeric keyboard-and-monitor video display terminals (VDTs).
- **Seminal Architectures:**
 - **IBM System/360 (1964):** A monumental **\$5 billion** engineering effort that unified disparate commercial and scientific architectures into a single scalable family of software-compatible machines, establishing the standard 8-bit byte and 32-bit word.
 - **DEC PDP-8 (1965):** Engineered by Digital Equipment Corporation, the PDP-8 was the world's first commercially triumphant **Minicomputer**, bringing computing out of specialized air-conditioned data centers directly into university laboratories and manufacturing floors at a fraction of mainframe costs.

Fourth Generation (1971–Present): Microprocessors, VLSI & The Personal Computer

The defining technological milestone of the fourth generation was the invention of the **Microprocessor** — the complete fabrication of a computer's Central Processing Unit (ALU, Control Unit, and internal registers) onto a single silicon semiconductor chip.

- **The Intel 4004 & Microprocessor Evolution:** In 1971, engineers Ted Hoff, Federico Faggin, Stan Mazor, and Masatoshi Shima designed the **Intel 4004** (a 4-bit microprocessor comprising **2,300** transistors on a **12 mm²** die running at **740 kHz**). This was rapidly followed by the 8-bit Intel 8008, the 8080 (1974), and the legendary 16-bit Intel 8086 (1978), which established the enduring x86 instruction set architecture.
- **Integration Density Scaling:** Semiconductor manufacturing progressed through **Large Scale Integration (LSI, 1,000 to 20,000 transistors)** to **Very Large Scale Integration (VLSI, 20,000 to 1,000,000+ transistors)**. **Moore's Law** — the empirical observation formulated by Gordon Moore in 1965 stating

- that the number of transistors on a microchip doubles approximately every 18 to 24 months while the cost per transistor halves — served as the driving roadmap for the microelectronics industry for half a century.
- **The Microcomputer & PC Revolution:** Microprocessors democratized computing. The MITS Altair 8800 (1975) sparked the home hobbyist movement, leading to Steve Wozniak and Steve Jobs creating the Apple II (1977), the Commodore PET, and the historic **IBM Personal Computer (Model 5150)** in August 1981, powered by the Intel 8088 CPU and Microsoft MS-DOS.
 - **Graphical User Interfaces & Networking:** The fourth generation witnessed the transition from command-line interfaces to **Graphical User Interfaces (GUI)** with the WIMP (Windows, Icons, Menus, Pointers) paradigm (pioneered at Xerox PARC and popularized by the Apple Macintosh in 1984 and Microsoft Windows). Simultaneously, local networking (Ethernet) and the global interconnected network of networks — the **Internet** (governed by the TCP/IP protocol suite) and the World Wide Web (invented by Tim Berners-Lee in 1989) — transformed computers from isolated calculating tools into global communications appliances.

Fifth Generation (Present & Emerging Future): ULSI, AI & Quantum Paradigms

The fifth generation transcends classical transistor dimensional scaling, moving into the realm of **Ultra Large Scale Integration (ULSI)** (chips exceeding **100 billion** transistors), massively parallel multi-core architectures, specialized hardware accelerators for Artificial Intelligence, and non-von Neumann computing systems.

- **Key Pillars:** Artificial Intelligence (AI), Machine Learning (ML), Deep Neural Network training and inference, Natural Language Processing (NLP), Computer Vision, and Autonomous Robotics.
- **Specialized Silicon Accelerators:** Massive deployment of Graphics Processing Units (GPUs), Tensor Processing Units (TPUs), and Neural Processing Units (NPU) engineered with dedicated matrix-multiplication systolic arrays.
- **Emerging Paradigms:** Quantum Processing Units (QPUs) exploiting quantum superposition and entanglement, Neuromorphic Computing (silicon spiking neural networks mimicking biological synapses), and Optical/Photonic Computing utilizing photons for speed-of-light data transmission.

1.3 Comprehensive Generational Comparison Table

Generation	Active Switching Element	Primary Memory Technology	Operating Speed	Software / OS Architecture	Typical Input / Output	Seminal Computing Machines
1st Gen (1940–56)	Thermionic Vacuum Tubes / Valves	Acoustic Delay Lines, Magnetic Drums	Milliseconds (10⁻³ s) ~thousands ops/sec	Raw Machine Language, Plugboards, No OS	Punched cards, paper tape, magnetic tape	ENIAC, EDVAC, EDSAC, UNIVAC I, IBM 701
2nd Gen (1956–63)	Discrete Bipolar Transistors (BJT)	Magnetic Ferrite Core Memory	Microseconds (10⁻⁶ s) ~hundreds of thousands ops/sec	Assembly Language, FORTRAN, COBOL, Early Batch Monitors	Punched cards, high-speed magnetic tape, early disk	IBM 7094, IBM 1401, CDC 1604, UNIVAC 1107
3rd Gen (1964–71)	Integrated Circuits (SSI & MSI on silicon)	Semiconductor RAM chips (DRAM), Core	Nanoseconds (10⁻⁹ s) ~millions ops/sec (MIPS)	Time-Sharing OS, Multiprogramming (OS/360, MULTICS, UNIX)	Keyboards, Alphanumeric Video Display Terminals (VDTs)	IBM System/360, DEC PDP-8, PDP-11, CDC 6600
4th Gen (1971–Pres.)	Microprocessors (LSI, VLSI, ULSI)	Semiconductor DRAM, SRAM, NVMe Flash SSDs	Picoseconds (10⁻¹² s) ~billions ops/sec (GFLOPS/TFLOPS)	GUI Operating Systems (Windows, Linux, macOS), C++, Java, Python	Mouse, Optical Keyboards, Touchscreens, 4K Monitors	Intel Core, AMD Ryzen, Apple Silicon M-Series, IBM PC
5th Gen (Emerging)	ULSI, Quantum Gates, 3D Chiplets, NPUs	High Bandwidth Memory (HBM3), Quantum States	Massively Parallel / ExaFLOPS (10¹⁸ ops/sec)	AI Frameworks (PyTorch, TensorFlow), Qiskit, Quantum OS	Natural Voice Interfaces, Neural BCI, Computer Vision	NVIDIA H100/B200, IBM Quantum Condor, Google Sycamore

Unit 2: Overview of Computer System — Von Neumann Model & Number Systems

2.1 The Von Neumann Computer Architecture

In June 1945, the brilliant Hungarian-American mathematician and polymath John von Neumann authored a landmark report entitled *"First Draft of a Report on the EDVAC"*. In this manuscript, von Neumann articulated an architectural blueprint for a general-purpose electronic computer that separated the machine's physical hardware from its logical programming.

Prior to the von Neumann model, early machines like the ENIAC were "fixed-program" or hardwired computers; if an engineer wished to switch from calculating an artillery trajectory to computing a nuclear hydrodynamic equation, the machine had to be powered down, manually rewired with thousands of patch cords, and physically reconfigured over several days.

The revolutionary insight of the **Stored-Program Concept** is that:

THE STORED-PROGRAM PRINCIPLE

Both program instructions (the operational algorithms) and the computational data (the operands) are encoded in identical binary format and stored together in the same shared, read-write primary memory space. The CPU sequentially fetches an instruction from memory, interprets its operation code (opcode), executes the corresponding micro-operations on data fetched from memory, and writes results back to memory using a unified bus interconnect.

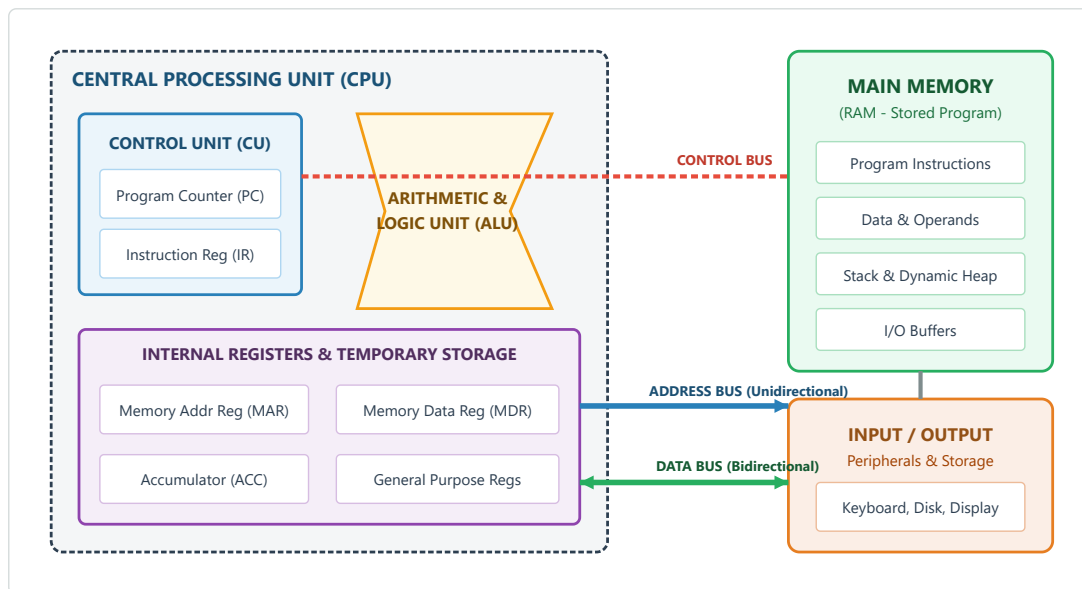


Figure 1.1: Complete Von Neumann Architecture with CPU Subsystems, Main Memory, and Tri-Bus Interconnect.

Detailed Subsystems of the Von Neumann Model

- The Central Processing Unit (CPU):** The computational brain of the system, comprising:
 - The Control Unit (CU):** The master coordinator and traffic controller of the computer. The CU generates synchronization clock pulses and control signals that orchestrate the fetching, decoding, and execution of instructions. It directs the movement of data between memory, ALU, internal registers, and input/output modules.
 - The Arithmetic and Logic Unit (ALU):** The functional unit that executes all computational operations:
 - Arithmetic Operations:** Addition, subtraction, multiplication, and division on binary integers and floating-point numbers.
 - Logical Operations:** Bitwise boolean operations (AND, OR, NOT, XOR, NOR, NAND) and comparison evaluations ($<$, $\leq$, $>$, $\geq$, $==$, $\neq$).
 - Bit-Shifting Operations:** Logical left/right shift, arithmetic right shift (preserving sign bits), and cyclic rotation.
 - The Internal CPU Register File:** Ultra-fast, low-capacity storage cells fabricated directly inside the CPU using static flip-flop circuits:
 - Program Counter (PC):** Holds the exact physical memory address of the *next* sequential instruction to be fetched from memory. It automatically increments after each fetch, unless redirected by a branch, jump, or subroutine call.
 - Instruction Register (IR):** Holds the binary opcode and operand specifiers of the instruction currently undergoing decoding and execution.
 - Memory Address Register (MAR):** Holds the physical memory address destined to be driven across the Address Bus during a memory read or write cycle.
 - Memory Buffer Register (MBR) / Memory Data Register (MDR):** Holds the actual data word that has just been read from memory or is waiting to be written to memory across the Data Bus.
 - Accumulator (ACC):** A primary general-purpose working register that holds the immediate operands and intermediate mathematical results generated by the ALU.
 - Program Status Word (PSW) / Flags Register:** A collection of individual status flip-flops (Zero Flag Z , Carry Flag C , Sign/Negative Flag S , Overflow Flag V , Parity Flag P , Interrupt Enable Flag I) updated by the ALU after each mathematical operation.
- The Main Memory Subsystem (RAM):** A linear, contiguous array of addressable storage cells (bytes). Each storage cell is assigned a unique non-negative integer address ranging from 0 to $2^N - 1$, where N represents the bit-width of the memory address bus. In the von Neumann model, instructions and data reside in this single uniform address space.
- The Input/Output (I/O) Subsystem:** Interfaces the high-speed internal CPU and memory with external human and machine peripherals (keyboards, monitors, network transceivers, secondary disk drives). It performs signal voltage level translation, serial-to-parallel data conversion, protocol handshaking, and buffering.
- The System Bus Interconnection Subsystem:** The shared electrical parallel highway connecting the CPU, Memory, and I/O, divided into three specialized functional buses:
 - Address Bus (Unidirectional):** Transmits memory and I/O port addresses originating exclusively from the CPU to external memory chips and controllers. An N -bit address bus can address 2^N distinct bytes of physical memory (e.g., a 32-bit address bus addresses $2^{32} = 4$ GB; a 64-bit address bus addresses $2^{64} = 16$ Exabytes).

- **Data Bus (Bidirectional):** Carries bidirectional data words and instruction opcodes between the CPU, memory, and I/O devices. The width of the data bus (W bits, e.g., 64 bits) determines the system's fundamental word transfer capacity per clock cycle.
- **Control Bus (Bidirectional):** Carries command, timing, and synchronization signals across the system (e.g., Memory Read **MEMR**, Memory Write **MEMW**, I/O Read **IOR**, I/O Write **IOW**, System Clock **CLK**, Interrupt Request **INTR**, Interrupt Acknowledge **INTA**, Bus Request **BREQ**, and Ready signals).

THE VON NEUMANN BOTTLENECK & THE MEMORY WALL

Because both instruction fetches and data read/writes must share the **same physical bus interconnect** in the von Neumann architecture, the CPU cannot read an instruction and read/write operand data at the exact same clock cycle.

Over decades, CPU processing frequencies grew exponentially (**50%** per year), whereas DRAM memory access latency improved at a sluggish pace (**7%** per year). This divergence created the **Memory Wall** — the CPU spends a substantial percentage of its clock cycles idle (in "wait states") waiting for data transfers across the system bus.

Mitigations in Modern Computer Architecture:

- **Multi-Level On-Die Cache Hierarchies (L1, L2, L3):** Placing ultra-high-speed SRAM caches directly onto the CPU die to satisfy up to **95%** of memory requests within 1 to 10 clock cycles.
- **Modified Harvard Architecture:** Implementing physically separated, independent L1 Instruction Cache (L1 I-Cache) and L1 Data Cache (L1 D-Cache) inside the CPU core, allowing simultaneous instruction fetch and data access, while maintaining a unified memory model in DRAM.
- **Hardware Branch Prediction & Speculative Execution:** Predicting execution paths ahead of time and prefetching data into caches before instructions are reached.

The Formal Four-Phase Instruction Execution Cycle

The CPU executes continuous programs by repeatedly cycling through four deterministic phases:

1. Fetch Phase:

- The content of the Program Counter (PC) is transferred to the Memory Address Register: $MAR \leftarrow PC$.
- The Control Unit asserts the **MEMR** signal on the control bus.
- The memory subsystem reads the instruction located at address MAR and places it onto the Data Bus, which is latched into the Memory Data Register: $MDR \leftarrow M[MAR]$.
- The instruction is copied from MDR to the Instruction Register: $IR \leftarrow MDR$.
- The Program Counter is automatically incremented to point to the next instruction in memory: $PC \leftarrow PC + instruction_size$.

2. Decode Phase:

- The Control Unit decodes the opcode bits stored in the IR using internal instruction decoders.
- Effective memory addresses of any required operands are calculated (resolving direct, indirect, register, or indexed addressing modes).

3. Execute Phase:

- If operands reside in memory, they are fetched via $MAR \leftarrow effective_address$ and $MDR \leftarrow M[MAR]$.
- The Control Unit signals the ALU to perform the specified arithmetic or logical operation (e.g., adding MDR to the Accumulator $ACC \leftarrow ACC + MDR$).
- If the instruction is a conditional branch (e.g., **JZ** — Jump if Zero) and the Zero Flag $Z = 1$, the PC is updated: $PC \leftarrow target_address$.

4. Store / Write-Back Phase:

- The final computed result in the Accumulator or working register is written back to the destination general-purpose register or to primary memory: $MAR \leftarrow dest_address, MDR \leftarrow result, M[MAR] \leftarrow MDR$.
- The flags register (PSW) is updated to reflect negative, zero, carry, or overflow conditions.

2.2 Positional Number Systems in Digital Computing

Digital computers are electronic switching machines. Because semiconductor transistors operate with maximum noise immunity when switching between two stable saturation states (cut-off = **0V** / Logic **0**, and saturation = **+1.8V** or **+3.3V** / Logic **1**), all computer arithmetic is founded upon positional number systems.

In a positional number system with base (radix) r , a number N possessing n integer digits and m fractional digits is represented as:

$$N = (d_{n-1}d_{n-2} \dots d_1d_0.d_{-1}d_{-2} \dots d_{-m})_r$$

where each digit d_i satisfies $0 \leq d_i < r$. The precise decimal magnitude of the number is calculated via polynomial expansion:

$$V(N) = \sum_{i=-m}^{n-1} d_i \cdot r^i = d_{n-1}r^{n-1} + d_{n-2}r^{n-2} + \dots + d_1r^1 + d_0r^0 + d_{-1}r^{-1} + \dots + d_{-m}r^{-m}$$

The Four Primary Number Systems

Number System	Radix (r)	Permissible Digits / Symbols	Positional Weight Multipliers	Primary Utility in Computing
Binary	r = 2	0, 1	..., 2 ³ = 8, 2 ² = 4, 2 ¹ = 2, 2 ⁰ = 1 · 2 ⁻¹ = 0.5, 2 ⁻² = 0.25	Physical hardware storage, logic gates, ALU arithmetic
Octal	r = 8	0, 1, 2, 3, 4, 5, 6, 7	..., 8 ² = 64, 8 ¹ = 8, 8 ⁰ = 1 · 8 ⁻¹ = 0.125, 8 ⁻² = 0.015625	Unix/Linux file permissions (e.g. <code>chmod 777</code>), legacy minicomputers
Decimal	r = 10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	..., 10 ² = 100, 10 ¹ = 10, 10 ⁰ = 1 · 10 ⁻¹ = 0.1, 10 ⁻² = 0.01	Human interface, accounting, financial and business software
Hexadecimal	r = 16	0–9, A(10), B(11), C(12), D(13), E(14), F(15)	..., 16 ² = 256, 16 ¹ = 16, 16 ⁰ = 1 · 16 ⁻¹ = 0.0625	Memory address notation (<code>0x7FFF80</code>), IPv6, MAC addresses, RGB colors

Unit 3: Number Conversions & Digital Codes

3.1 Systematic Base Conversion Procedures

Translating numbers between bases is an essential mathematical operation in computer science. Integer and fractional components are converted using complementary, inverse algorithms.

1. Decimal to Any Base r

- **Integer Part:** Apply repeated successive division by target radix **r**. In each step, record the integer quotient and the remainder ($0 \leq \text{remainder} < r$). Repeat until the quotient reaches **0**. The converted integer is formed by collecting the remainders in **reverse chronological order (from bottom to top, representing Most Significant Digit to Least Significant Digit)**.
- **Fractional Part:** Apply repeated successive multiplication by target radix **r**. In each step, record the resulting integer digit and retain the new fractional remainder for subsequent multiplication. Terminate when the fraction reaches exact **0.0** or when the required degree of precision (number of fractional digits) is attained. The converted fraction is formed by collecting the integer digits in **direct chronological order (from top to bottom)**.

WORKED EXAMPLE 3.1: DECIMAL TO BINARY & HEXADECIMAL

Problem: Convert decimal number $(109.6875)_{10}$ into Binary and Hexadecimal.

Step 1: Decimal to Binary

Integer Part $(109)_{10}$:

- $109 \div 2 = 54$ with remainder 1 (LSD b_0)
- $54 \div 2 = 27$ with remainder 0
- $27 \div 2 = 13$ with remainder 1
- $13 \div 2 = 6$ with remainder 1
- $6 \div 2 = 3$ with remainder 0
- $3 \div 2 = 1$ with remainder 1
- $1 \div 2 = 0$ with remainder 1 (MSD b_6)

Reading bottom-to-top: $(109)_{10} = (1101101)_2$.

Fractional Part $(0.6875)_{10}$:

- $0.6875 \times 2 = 1.3750 \Rightarrow$ Record 1, fraction remains 0.3750
- $0.3750 \times 2 = 0.7500 \Rightarrow$ Record 0, fraction remains 0.7500
- $0.7500 \times 2 = 1.5000 \Rightarrow$ Record 1, fraction remains 0.5000
- $0.5000 \times 2 = 1.0000 \Rightarrow$ Record 1, fraction is 0.0000 (Done)

Reading top-to-bottom: $(0.6875)_{10} = (0.1011)_2$.

Combined Binary Result: $(109.6875)_{10} = (1101101.1011)_2$.

Step 2: Decimal to Hexadecimal

Integer Part $(109)_{10}$:

- $109 \div 16 = 6$ remainder 13 $\Rightarrow$ D
- $6 \div 16 = 0$ remainder 6

Reading bottom-to-top: $(109)_{10} = (6D)_{16}$.

Fractional Part $(0.6875)_{10}$:

- $0.6875 \times 16 = 11.0000 \Rightarrow$ Record 11 $\Rightarrow$ B (Done)

Combined Hexadecimal Result: $(109.6875)_{10} = (6D.B)_{16}$.

2. Fast Direct Grouping: Binary $\leftrightarrow$ Octal $\leftrightarrow$ Hexadecimal

Because the bases are powers of 2 ($8 = 2^3$ and $16 = 2^4$), conversion between binary and octal or hexadecimal requires no arithmetic calculations — only simple bit clustering:

- **Binary to Octal:** Group bits into clusters of 3 starting from the radix point (move left for integer part, padding leading zeros; move right for fractional part, padding trailing zeros). Replace each 3-bit group with its corresponding octal digit (0–7).
- **Binary to Hexadecimal:** Group bits into clusters of 4 starting from the radix point (pad zeros left and right as needed). Replace each 4-bit nibble with its hexadecimal symbol (0–9, A–F).
- **Hexadecimal to Octal:** Convert each hexadecimal digit into its exact 4-bit binary equivalent, and then regroup the combined binary string into 3-bit octal clusters.

WORKED EXAMPLE 3.2: GROUPING CONVERSIONS

Convert $(10110110101.110101)_2$ to Octal and Hexadecimal:

To Octal (3-bit clusters):

$$\underbrace{010}_2 \underbrace{110}_6 \underbrace{110}_6 \underbrace{101}_5 \cdot \underbrace{110}_6 \underbrace{101}_5 \Rightarrow (2665.65)_8$$

To Hexadecimal (4-bit clusters):

$$\underbrace{0101}_5 \underbrace{1011}_B \underbrace{0101}_5 \cdot \underbrace{1101}_D \underbrace{0100}_4 \Rightarrow (5B5.D4)_{16}$$

3.2 Complements & Signed Binary Arithmetic

In digital processors, subtraction is executed using addition hardware by encoding negative numbers in **Complements**.

- **1's Complement:** Formed by bitwise inverting all bits ($0 \rightarrow 1, 1 \rightarrow 0$). For an n -bit number N , $1's \text{ Comp}(N) = (2^n - 1) - N$. Disadvantages: dual representations for zero ($+0 = 0000_2, -0 = 1111_2$) and requires an "end-around carry" addition stage.
- **2's Complement:** The universal standard in modern computing architectures. Formed by adding 1 to the 1's complement:

$$2's \text{ Comp}(N) = [1's \text{ Comp}(N)] + 1 = 2^n - N$$

Key Properties:

- Provides a single unique representation for zero (00000000_2).
- An n -bit 2's complement register represents signed integers in the range $-2^{n-1} \leq N \leq +2^{n-1} - 1$ (e.g., 8-bit signed range is -128 to $+127$).
- Subtractions are computed via simple binary addition: $A - B = A + [2's \text{ Comp}(B)]$. Any final carry-out bit beyond the n -th bit is simply discarded.

WORKED EXAMPLE 3.3: 2'S COMPLEMENT SUBTRACTION

Evaluate $(45)_{10} - (23)_{10}$ using 8-bit 2's complement arithmetic:

$$+45 = (00101101)_2$$

$$+23 = (00010111)_2$$

$$1's \text{ Complement of } 23 = 11101000_2$$

$$2's \text{ Complement of } 23 = 11101000_2 + 1 = (11101001)_2$$

Now execute binary addition:

$$\begin{array}{r} 00101101 \quad (+45) \\ +11101001 \quad (-23 \text{ in } 2's \text{ complement}) \\ \hline 1\ 00010110 \quad (\text{Final carry 1 is discarded}) \end{array}$$

The result is $(00010110)_2 = 16 + 4 + 2 = (22)_{10}$, which is correct ($45 - 23 = 22$).

3.3 Digital Codes

Digital codes are structured binary representations engineered for computation, data transmission, or electromechanical sensing.

1. Binary Coded Decimal (BCD / 8421 Code)

BCD represents each decimal digit (0 through 9) independently using a 4-bit binary nibble with weights **8, 4, 2, 1**.

- Decimal $(852)_{10}$ in BCD is: $\underbrace{1000}_8 \underbrace{0101}_5 \underbrace{0010}_2$.
- **Invalid BCD States:** A 4-bit nibble can represent 16 states (0000_2 to 1111_2). States 1010_2 (10_{10}) through 1111_2 (15_{10}) are **illegal/invalid BCD codes**.
- **BCD Addition Correction Rule:** When adding two BCD digits, if the sum exceeds **9** (1001_2) or generates an internal binary carry, the result contains an invalid code. To correct the error and propagate a proper decimal carry, add $(0110)_2$ (6_{10}) to that specific 4-bit sum.

2. Excess-3 Code (XS-3)

Excess-3 is an **unweighted, self-complementing** decimal code formed by adding $(0011)_2$ (3_{10}) to the 8421 BCD representation of each digit.

- **Self-Complementing Characteristic:** The 1's complement (bitwise NOT) of the Excess-3 code of digit d directly yields the Excess-3 representation of its 9's complement ($9 - d$). For instance, the Excess-3 code for **2** is 0101_2 ; inverting all bits yields 1010_2 , which is the Excess-3 code for **7** ($9 - 2 = 7$). This property greatly simplified subtraction circuitry in early computing ALUs.

3. Gray Code (Reflected Binary Code)

Gray code is an **unweighted, unit-distance** binary code in which any two successive numerical values differ in exactly **one single bit position**.

- **Application in Shaft Encoders:** When an optical or mechanical shaft angle encoder rotates between positions, standard binary transitions (such as $3 = 011_2 \rightarrow 4 = 100_2$) require multiple bits to change simultaneously. Due to microscopic mechanical tolerances, switches never change at the exact same microsecond, causing transient intermediate reading errors (e.g., momentarily reading $111_2 = 7$). Gray code guarantees that only one bit changes per angular increment, completely eliminating reading glitches.
- **Binary to Gray Conversion Algorithm:** Let Binary be $B = b_{n-1}b_{n-2} \dots b_0$ and Gray be $G = g_{n-1}g_{n-2} \dots g_0$:

$$g_{n-1} = b_{n-1} \quad (\text{MSB is preserved})$$

$$g_i = b_{i+1} \oplus b_i \quad \text{for } i = n-2, \dots, 0$$

- **Gray to Binary Conversion Algorithm:**

$$b_{n-1} = g_{n-1}$$

$$b_i = b_{i+1} \oplus g_i \quad \text{for } i = n-2, \dots, 0$$

WORKED EXAMPLE 3.4: GRAY CODE CONVERSIONS

1. Convert Binary $(1011)_2$ to Gray Code:

- $g_3 = b_3 = 1$
- $g_2 = b_3 \oplus b_2 = 1 \oplus 0 = 1$
- $g_1 = b_2 \oplus b_1 = 0 \oplus 1 = 1$
- $g_0 = b_1 \oplus b_0 = 1 \oplus 1 = 0$

Result: $(1011)_2 = (1110)_{\text{Gray}}$.

2. Convert Gray Code $(1110)_{\text{Gray}}$ back to Binary:

- $b_3 = g_3 = 1$
- $b_2 = b_3 \oplus g_2 = 1 \oplus 1 = 0$
- $b_1 = b_2 \oplus g_1 = 0 \oplus 1 = 1$
- $b_0 = b_1 \oplus g_0 = 1 \oplus 0 = 1$

Result: $(1110)_{\text{Gray}} = (1011)_2$.

Unit 4: Pioneers & Contributors of Computing Systems

The architectural and theoretical edifices of computer science are the direct culmination of seminal discoveries made by visionary thinkers. This section details the explicit contributions of the five monumental pioneers specified in the syllabus.

4.1 Charles Babbage (1791–1871) — The First Mechanical Computer

Celebrated historically as the "*Father of the Computer*", British mathematician, mechanical engineer, and polymath Charles Babbage conceived the earliest programmable computing machines during the 19th century.

- **The Difference Engine (1822):** Babbage was deeply frustrated by the pervasive human errors in printed navigation, astronomical, and logarithmic tables. He engineered the Difference Engine — a massive mechanical special-purpose calculator that computed polynomial functions automatically using the mathematical **method of finite differences**. By eliminating multiplication and division in favor of pure repeated additions, the machine could tabulate mathematical tables and automatically imprint results onto stereotype printing plates.
- **The Analytical Engine (1837):** Babbage's crowning intellectual masterpiece — the world's first design for a general-purpose, programmable, fully automatic mechanical computer. The Analytical Engine incorporated every fundamental architectural component found in modern electronic systems:
 - **The "Store":** A mechanical memory unit capable of storing 1,000 numbers of 50 decimal digits each.
 - **The "Mill":** The arithmetic processing unit containing geared columns that executed addition, subtraction, multiplication, and division under mechanical control.
 - **Control & Program Input:** Utilized strings of Jacquard punched cards to input operational instructions and variable values. The machine supported conditional branching (if-then testing of mechanical sign levers) and looping.
 - **Output Unit:** An automated typesetting mechanism and curve plotter.
- **Ada Lovelace (1815–1852):** Augusta Ada King, Countess of Lovelace, translated Luigi Menabrea's French paper on Babbage's Analytical Engine, appending extensive "Notes". Note G contained an algorithm to compute Bernoulli numbers using the engine — recognized today as the **world's first computer program**. Lovelace made the profound philosophical leap that Babbage's engine could manipulate any entities governed by rules (including music and symbolic logic), establishing the concept of universal symbolic computation.

4.2 John von Neumann (1903–1957) — The Stored-Program Architecture

Hungarian-American mathematician John von Neumann revolutionized mathematics, quantum mechanics, nuclear physics, game theory, and computer systems design.

- **First Draft of a Report on the EDVAC (1945):** Von Neumann formally synthesized the **stored-program architecture**. He established the standard model comprising a Central Processing Unit (ALU, Control Unit, Register File), a unified primary memory holding both instructions and data, and Input/Output subsystems interconnected via buses.
- **The IAS Machine (1952):** Built at the Institute for Advanced Study in Princeton, the IAS computer served as the physical architectural prototype for the entire mainframe industry (including the IBM 701) and established the foundational 40-bit word and parallel arithmetic processing standards.
- **Theoretical Advances:** Authored foundational mathematical papers on self-reproducing cellular automata, stochastic computing, and reliable computing architectures constructed from unreliable components.

4.3 Alan Turing (1912–1954) — Theoretical Foundations of Computer Science

British mathematician, logician, and wartime cryptanalyst Alan Mathison Turing is universally revered as the "*Father of Theoretical Computer Science and Artificial Intelligence*".

- **The Universal Turing Machine (1936):** In his landmark paper "*On Computable Numbers, with an Application to the Entscheidungsproblem*", Turing formulated an abstract mathematical model of computation consisting of an infinite tape divided into discrete cells, a read/write head, and a state register

governed by a finite transition table. Turing proved that a single **Universal Turing Machine (UTM)** could simulate any algorithmic computation whatsoever if encoded as a program on its tape. He proved the fundamental mathematical limits of computation, demonstrating the unsolvability of the **Halting Problem** (it is undecidable whether an arbitrary program will terminate or run forever).

- **Wartime Cryptanalysis at Bletchley Park:** During World War II, Turing was the intellectual leader at Hut 8. He designed the electromechanical **Bombe** machine, which broke the German military Enigma cipher at industrial speed, decrypting millions of military communications (Ultra intelligence) that historians estimate shortened the war in Europe by two to four years, saving millions of lives.
- **The Turing Test for Artificial Intelligence (1950):** In "*Computing Machinery and Intelligence*", Turing proposed the definitive operational benchmark for machine intelligence (the *Imitation Game*), asserting that if a human interrogator communicating via text cannot reliably tell a machine from a human, the machine exhibits true intelligent behavior.

4.4 John Mauchly (1907–1980) & J. Presper Eckert (1919–1995) — ENIAC & UNIVAC

Physicist John Mauchly and electrical engineering genius J. Presper Eckert Jr. formed the legendary hardware engineering partnership at the University of Pennsylvania's Moore School.

- **ENIAC (1943–1945):** Conceived to calculate artillery firing tables for the US Army Ballistics Research Laboratory, ENIAC was the world's **first operational, general-purpose electronic digital computer**. Eckert's engineering brilliance ensured unprecedented vacuum tube reliability by operating tube heaters at conservative voltages, proving that colossal electronic machines could function reliably.
- **EDVAC & BINAC:** Co-designed EDVAC and engineered BINAC (Binary Automatic Computer), the first stored-program computer in the US to utilize magnetic tape storage.
- **UNIVAC I & The Commercial Computer Industry:** Founded the Eckert-Mauchly Computer Corporation (EMCC). In 1951, they delivered **UNIVAC I**, marking the birth of the commercial mainframe industry.

4.5 John McCarthy (1927–2011) — The Father of Artificial Intelligence

American computer scientist and cognitive scientist John McCarthy made monumental contributions that shaped programming languages, operating systems, and intelligent machines.

- **Coining the Term "Artificial Intelligence" (1955):** McCarthy coined the phrase *Artificial Intelligence* in his formal proposal for the historic 1956 **Dartmouth Summer Research Project on Artificial Intelligence**, establishing AI as an autonomous, formal academic discipline.
- **Creation of LISP (1958):** Invented **LISP (List Processing)**, the second-oldest high-level programming language in continuous use. LISP introduced revolutionary concepts: dynamic memory structures, symbolic computation, recursive functions, higher-order functions, and abstract syntax trees.
- **Invention of Automatic Garbage Collection (1959):** To manage dynamic memory allocation in LISP without manual pointer reclamation, McCarthy invented **Garbage Collection** — the memory safety mechanism now universal in Java, Python, JavaScript, and C#.
- **Time-Sharing Operating Systems:** Pioneered the concept of computer time-sharing at MIT (CTSS), advocating for utility computing where thousands of remote users could interact with a central computing pool simultaneously.

Unit 5: Computing Systems: Past to Present Architectures

5.1 Processor Micro-Architectural Evolution

Microprocessor design has undergone a radical architectural transformation over the past five decades, driven by semiconductor physics and the pursuit of extreme parallel throughput.

1. Single-Core Processors & Frequency Scaling Limits

Early microprocessors contained a single physical execution engine that executed instructions in strict serial order. Designers scaled single-core performance through:

- **Instruction Pipelining:** Dividing the instruction cycle into discrete hardware stages (Fetch, Decode, Execute, Memory Access, Write-Back), enabling multiple instructions to be processed simultaneously at different stages like an assembly line.
- **Superscalar Execution:** Deploying multiple parallel execution units (ALUs, FPUs) to dispatch and complete multiple instructions per clock cycle ($IPC > 1$).
- **Out-of-Order (OoO) Execution:** Dynamically reordering instructions based on operand availability to avoid pipeline stalls.
- **The Breakdown of Dennard Scaling & The Power Wall (circa 2005):** Under classical **Dennard Scaling**, as transistor dimensions shrank by a factor of k , capacitance dropped and operating voltage could be scaled down proportionally, keeping power density constant ($P = CV^2f$). However, below **90 nm**, quantum tunneling leakage currents through atomic-scale gate oxides exploded. Operating voltage could no longer be reduced safely without compromising transistor stability. CPU power dissipation and thermal heat flux increased exponentially, hitting the **Power Wall** at ~4 GHz clock speeds, halting the single-core frequency race.

2. Dual-Core and Multi-Core Processors (CMP)

To continue performance scaling within acceptable thermal budgets (Thermal Design Power — TDP), the industry transitioned from monolithic single-core designs to **Chip Multiprocessors (CMP)** — integrating two, four, eight, or dozens of independent processing cores onto a single silicon die.

- **Thread-Level Parallelism (TLP):** Multiple execution threads execute concurrently across distinct physical CPU cores.
- **Simultaneous Multithreading (SMT) / Hyper-Threading:** Duplicating architectural register state (PC, registers) on a single physical core, presenting two logical processors to the operating system to maximize execution engine utilization during memory stalls.
- **Hierarchical Cache Coherence:** In multi-core processors, each core maintains private low-latency L1 and L2 caches, while sharing a massive Last-Level Cache (L3 Cache). When multiple cores hold copies of the same shared main memory location, hardware **Cache Coherence Protocols (MESI — Modified,**

Exclusive, Shared, Invalid, and MOESI) snoop the on-chip ring bus or mesh network to invalidate or update stale cache lines automatically.

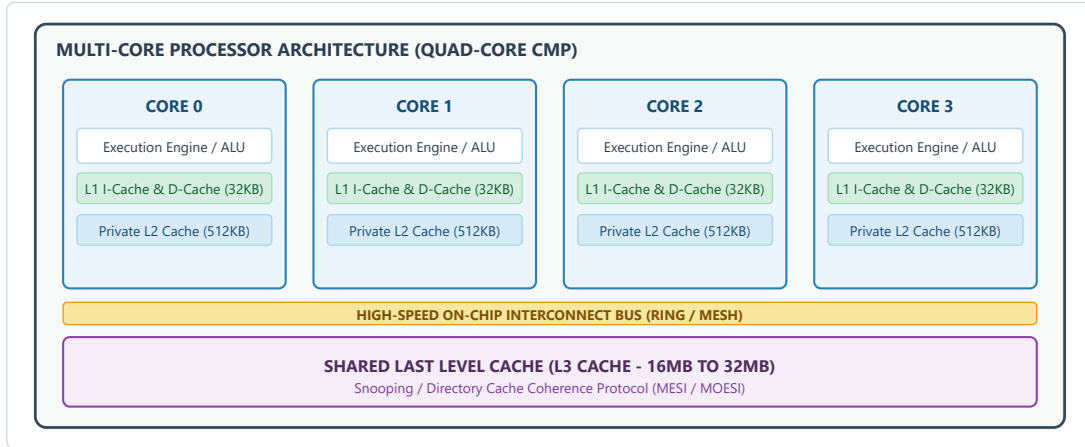


Figure 1.2: Multi-Core Chip Multiprocessor (CMP) Architecture with Multi-Level Cache Hierarchy.

5.2 Graphics Processing Units (GPUs) & SIMD/SIMT Architecture

While a CPU is a latency-optimized processor engineered with massive caches and branch predictors to execute complex, sequential control flow with minimal delay, a **Graphics Processing Unit (GPU)** is a throughput-optimized processor designed for massive data-parallel computational workloads.

- **Architectural Organization:** A modern GPU (e.g., NVIDIA Hopper/Blackwell, AMD RDNA) contains thousands of compact, power-efficient Arithmetic Logic Units (ALUs) grouped into hundreds of Streaming Multiprocessors (SMs).
- **Execution Model:** Operates on the **SIMD (Single Instruction, Multiple Data)** and **SIMT (Single Instruction, Multiple Threads)** models. A single instruction is broadcast and executed simultaneously across thousands of parallel threads operating on distinct vector/matrix data elements.
- **General-Purpose GPU (GPGPU) Computing:** The release of frameworks like NVIDIA CUDA and OpenCL transformed GPUs into the primary computational engines for Deep Learning, Large Language Model training, computer vision, molecular dynamics, and cryptography.

5.3 Accelerated Processing Units (APUs) & Unified Memory

An **Accelerated Processing Unit (APU)** (pioneered by AMD and conceptually mirrored in Apple Silicon M-Series Unified Memory SoCs) integrates high-performance multi-core CPU cores and a powerful GPU onto a single monolithic silicon die.

- **Heterogeneous Unified Memory Architecture (hUMA):** In conventional PC systems, data must be copied across the high-latency PCIe expansion bus between system RAM and GPU VRAM. An APU allows CPU and GPU cores to share a unified physical memory address space, eliminating costly memory copies and reducing latency.
- **Benefits:** Exceptional energy efficiency, compact physical form factors, and optimized graphics performance in laptops, handheld gaming systems (Steam Deck), and home consoles (PlayStation 5, Xbox Series X).

5.4 Quantum Processing Units (QPUs) — Fundamental Concepts

A **Quantum Processing Unit (QPU)** is a computational device that harnesses the bizarre principles of quantum mechanics to solve certain intractable mathematical problems exponentially faster than classical supercomputers.

Core Principles of Quantum Computation:

1. **The Qubit (Quantum Bit):** Unlike a classical bit which is deterministically either **0** or **1**, a qubit is a two-level quantum system represented using Dirac bra-ket notation as a linear combination (superposition):

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where $\alpha, \beta \in \mathbb{C}$ are complex probability amplitudes satisfying the normalization constraint:

$$|\alpha|^2 + |\beta|^2 = 1$$

Upon measurement, the quantum superposition collapses irreversibly to classical state $|0\rangle$ with probability $|\alpha|^2$ or to state $|1\rangle$ with probability $|\beta|^2$.

2. **Quantum Superposition:** An array of n entangled qubits can exist simultaneously in a superposition of all 2^n computational basis states:

$$|\Psi\rangle = \sum_{k=0}^{2^n-1} c_k |k\rangle$$

While n classical bits represent exactly *one* value at any instant, n qubits hold all 2^n combinations simultaneously (**300 qubits $\approx 2^{300}$ states**, vastly exceeding the number of atoms in the observable universe).

3. **Quantum Entanglement:** A uniquely quantum phenomenon where two or more qubits become intimately correlated such that the quantum state of one qubit cannot be described independently of the others, regardless of physical separation (e.g., Bell State $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$).
4. **Quantum Interference & Algorithms:** Quantum algorithms (such as **Shor's Algorithm** for polynomial-time integer factorization and **Grover's Algorithm** for $O(\sqrt{N})$ unstructured database search) utilize quantum logic gates (Hadamard H , Pauli-X, CNOT) to constructively amplify the probability amplitudes of correct solutions while destructively cancelling erroneous paths.

Classical Computers vs Quantum Processors (QPUs)

Feature / Metric	Classical Computer (CPU / GPU)	Quantum Processing Unit (QPU)
Fundamental Information Unit	Binary Bit (0 or 1)	Qubit ($\alpha 0\rangle + \beta 1\rangle$)
Governing Physical Laws	Classical Electrodynamics, CMOS voltage states	Quantum Mechanics (Superposition, Entanglement, Interference)
Operating Environment	Room temperature (20°C to 70°C)	Near absolute zero ($\approx 15\text{ mK}$) in cryogenic dilution refrigerators
Error Susceptibility	Virtually zero (10^{-17} soft error rate)	Extremely high susceptibility to environmental thermal decoherence
Optimal Problem Domains	General software, operating systems, databases, web	Molecular simulation, quantum chemistry, cryptography (RSA factoring)

Fundamentals of Computers and Computational Thinking

Course Code: CS101 Credits: 4 (Lecture: 3, Practical: 2) Total Hours: 75 Hrs

MODULE II: Hardware Components & System Architecture

Comprehensive Academic Notes — Units 6 through 11 (Duration: 11 Hours, Weightage: 20 Marks)

Unit 6: Electronic Components — Active Components

CONCEPTUAL DEFINITION: ACTIVE ELECTRONIC COMPONENT

An **active electronic component** is a physical circuit element that relies on an external power source to operate, possesses the physical ability to dynamically control and modulate the flow of charge carriers (electrons and holes), and is capable of introducing electrical power gain, signal amplification, or high-speed non-linear digital switching into an electronic circuit.

The hardware of all modern computing systems is constructed from solid-state semiconductor electronics. Semiconductor materials, predominantly **Silicon (Si)** (atomic number 14, possessing four valence electrons in its outer shell) and occasionally **Germanium (Ge)** or **Gallium Arsenide (GaAs)**, form crystalline tetrahedral covalent lattice structures. In its pure (intrinsic) state, silicon is an insulator at absolute zero and a poor conductor at room temperature (**300 K**) because very few thermal electron-hole pairs exist in the conduction band.

Through the controlled introduction of minute chemical impurities — a process termed **doping** (typically 1 impurity atom per 10^6 to 10^8 silicon atoms) — the electrical conductivity of silicon can be increased by many orders of magnitude:

- **N-Type Semiconductor (Negative Extrinsic Silicon):** Formed by doping intrinsic silicon with pentavalent donor atoms (such as Phosphorus **P**, Arsenic **As**, or Antimony **Sb**, possessing 5 valence electrons). Four of the valence electrons form strong covalent bonds with adjacent silicon atoms in the crystal lattice, leaving the fifth valence electron loosely bound with an energy level just below the conduction band (**0.05 eV**). At room temperature, this donor electron is readily promoted to the conduction band. Consequently, **free mobile electrons act as majority charge carriers**, while thermally generated holes serve as minority carriers.
- **P-Type Semiconductor (Positive Extrinsic Silicon):** Formed by doping intrinsic silicon with trivalent acceptor atoms (such as Boron **B**, Aluminum **Al**, or Gallium **Ga**, possessing 3 valence electrons). The deficiency of one valence electron creates a localized vacancy or **hole** in the covalent bond structure. An electron from an adjacent bond can easily hop into this vacancy, causing the positive hole to propagate through the lattice. Consequently, **mobile positive holes act as majority charge carriers**, while free electrons serve as minority carriers.

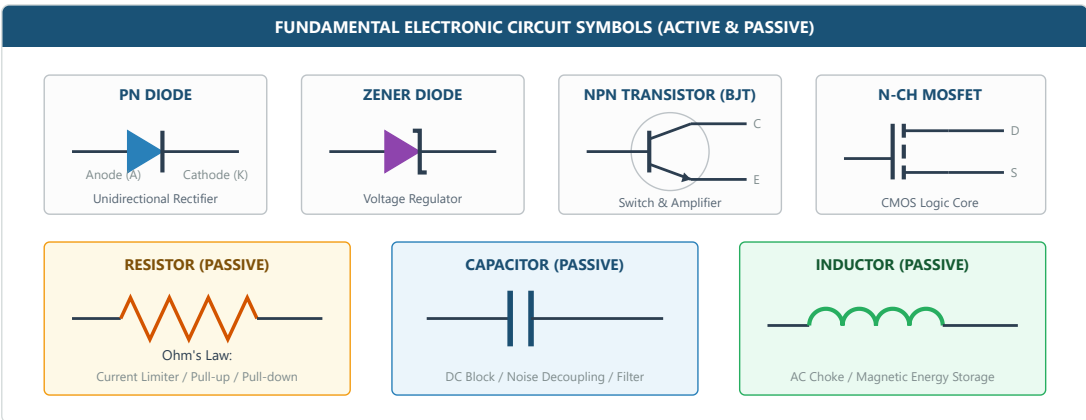


Figure 2.1: Active Semiconductor Components (Diodes, Transistors, ICs) and Passive Components (Resistors, Capacitors, Inductors).

6.1 Semiconductor Diodes

A **Diode** is a two-terminal, non-linear semiconductor device formed by metallurgically joining P-type and N-type semiconductor regions, creating an internal **PN Junction**.

1. Standard PN Junction Diode

- **Physical Operation & Barrier Potential:** Immediately upon PN junction formation, free electrons in the N-region diffuse across the junction to recombine with free holes in the P-region. This leaves behind fixed, uncompensated positive donor ions (**P⁺**) on the N-side and fixed negative acceptor ions (**B⁻**) on the P-side. This uncharged region containing fixed ions is termed the **depletion region**. The resulting internal electrostatic field creates a built-in barrier potential (**V₀ ≈ 0.7 V** for Silicon at **300 K**, and **V₀ ≈ 0.3 V** for Germanium).
 - **Forward Bias:** When an external DC voltage source is connected with positive polarity to the P-region (Anode) and negative to the N-region (Cathode), the applied potential opposes the built-in barrier. When **V > 0.7 V** (the knee/threshold voltage), the depletion layer collapses, and current increases exponentially according to the Shockley Diode Equation:

$$I = I_s \left(e^{\frac{qV}{\eta kT}} - 1 \right)$$

where I_s is reverse saturation current, q is electron charge (1.602×10^{-19} C), k is Boltzmann constant (1.38×10^{-23} J/K), T is absolute temperature in Kelvin, and η is ideality factor ($1 \leq \eta \leq 2$).

- **Reverse Bias:** Connecting positive polarity to N and negative to P widens the depletion layer, preventing majority carrier flow. Only a minuscule, nanoampere-level reverse saturation current (I_s) flows due to thermally generated minority carriers, acting as an open switch.
- **Functions in Computing Systems:**
 - **AC to DC Rectification:** Converting **230V/110V** AC power from mains into DC voltage in power supply units (Half-Wave, Full-Wave Bridge Rectifiers).
 - **Transient Voltage Suppression (TVS):** Clamping electrostatic discharge (ESD) spikes on external USB and HDMI port lines to protect delicate microchips.
 - **Polarity Protection:** Preventing reverse-current damage to sensitive motherboard circuits.

2. Specialized Diode Types

- **Zener Diode:** A heavily doped PN junction engineered to operate reliably in the **reverse breakdown region** (Zener and Avalanche breakdown). When reverse voltage reaches the specified Zener breakdown threshold (V_Z), the voltage across the diode remains virtually constant despite wide changes in reverse current.
Function: Precision voltage regulation and reference sources across motherboard VRMs.
- **Light Emitting Diode (LED):** Fabricated from direct bandgap compound semiconductors (such as Gallium Arsenide **GaAs** and Gallium Nitride **GaN**). Under forward bias, electrons and holes recombine directly across the bandgap, releasing discrete photons of light ($E = h\nu = \frac{hc}{\lambda}$). *Function:* Motherboard POST diagnostic debug LEDs, optical network transceivers, and drive activity indicators.
- **Photodiode:** A reverse-biased PN junction where incoming photons generate electron-hole pairs, producing a photocurrent proportional to light intensity.
Function: Fiber-optic optical receivers and ambient light sensors.

6.2 Transistors: BJT and MOSFET

The **Transistor** is the foundational solid-state switching and amplifying element of all modern digital electronics.

1. Bipolar Junction Transistor (BJT)

A current-controlled, three-terminal semiconductor device consisting of two back-to-back PN junctions, constructed in either **NPN** or **PNP** configurations.

- **Terminals:**
 - **Emitter (E):** Heavily doped to inject maximum charge carriers into the base.
 - **Base (B):** Physically ultra-thin and lightly doped, controlling the transit of carriers.
 - **Collector (C):** Moderately doped and physically largest to dissipate thermal heat while collecting charge carriers.
- **Operational States:**
 - **Cut-Off State ($V_{BE} < 0.7V$):** Both emitter-base and collector-base junctions are reverse-biased. Collector current $I_C \approx 0$. Functions as an **Open Digital Switch (Logic 0)**.
 - **Saturation State:** Both junctions are forward-biased. Collector current reaches maximum value limited only by external circuit resistance ($V_{CE} \approx 0.2V$). Functions as a **Closed Digital Switch (Logic 1)**.
 - **Active Region:** Emitter-base forward-biased, collector-base reverse-biased. $I_C = \beta \cdot I_B$, where β is current amplification factor (**50–300**). Functions as an analog amplifier.

2. Metal-Oxide-Semiconductor Field-Effect Transistor (MOSFET)

A voltage-controlled, four-terminal field-effect device (Gate, Drain, Source, Body/Substrate) where the metallic/polysilicon gate is electrically isolated from the semiconductor substrate by an ultra-thin dielectric insulating layer of Silicon Dioxide (**SiO₂**) or High- κ dielectric material (Hafnium Oxide **HfO₂**).

- **N-Channel Enhancement MOSFET (NMOS):** Applying a positive voltage to the Gate exceeding threshold voltage ($V_{GS} > V_{th}$) creates an electrostatic field that attracts electrons to the surface of the P-substrate, forming an inverted conductive N-channel between Source and Drain. Drain current I_D flows freely. When $V_{GS} = 0$, the channel vanishes, cutting off current.
- **Complementary MOS (CMOS) Technology:** Modern microprocessors combine NMOS and PMOS transistors in symmetrical complementary networks. In CMOS digital logic gates (e.g., CMOS Inverter / NOT gate), when the input is high, the NMOS pull-down network conducts to ground while the PMOS pull-up network is OFF; when the input is low, the PMOS conducts to V_{CC} while the NMOS is OFF. Because no direct path to ground exists in steady state, CMOS circuits consume near-zero static DC power, dissipating energy almost exclusively during dynamic switching transitions:

$$P_{dyn} = C \cdot V_{DD}^2 \cdot f$$

6.3 Integrated Circuits (ICs)

An **Integrated Circuit (IC)** is a complete electronic circuit comprising millions to billions of interconnected microscopic transistors, diodes, resistors, and capacitors fabricated directly on a single monolithic semiconductor silicon crystal substrate using advanced photolithography, ion implantation, chemical vapor deposition, and plasma etching.

- **Scales of Integration:**
 - **SSI (Small Scale Integration):** **< 10 logic gates** (1960s, basic logic gates 7400 series).
 - **MSI (Medium Scale Integration):** **10 to 100 gates** (multiplexers, decoders, counters).
 - **LSI (Large Scale Integration):** **100 to 10,000 gates** (early 8-bit microprocessors, Intel 8080).
 - **VLSI (Very Large Scale Integration):** **10,000 to 1,000,000 gates** (16/32-bit CPUs, Intel 80386/80486).
 - **ULSI (Ultra Large Scale Integration):** **> 1,000,000 to 100 billion+ transistors** (modern CPUs, GPUs, NPUs).

- **IC Packaging:** Dual In-line Package (DIP), Quad Flat Package (QFP), Ball Grid Array (BGA), and Land Grid Array (LGA). Modern microchips utilize multi-die 2.5D/3D Chiplet packaging interconnected via silicon interposers (e.g., TSMC CoWoS, Intel EMIB).

Unit 7: Electronic Components — Passive Components

CONCEPTUAL DEFINITION: PASSIVE ELECTRONIC COMPONENT

A **passive electronic component** is a circuit element that does not require an external power supply to function, cannot introduce net electrical power amplification into a circuit, and operates by dissipating, storing, or releasing electrical energy based on fundamental physical and electromagnetic laws.

Passive components form the indispensable foundation of all digital computer hardware. Without passive components to filter high-frequency noise, limit current surges, terminate high-speed transmission lines, and stabilize power rails, active microprocessors and memory chips cannot operate reliably.

7.1 Resistors

A **Resistor** is a two-terminal passive component designed to introduce precise electrical resistance (R) into a circuit, opposing the flow of electric charge in accordance with **Ohm's Law**:

$$V = I \cdot R \implies I = \frac{V}{R} \implies R = \frac{V}{I}$$

The resistance is determined by material resistivity (ρ), length (L), and cross-sectional area (A):

$$R = \rho \frac{L}{A}$$

Electrical power dissipated as thermal heat is given by **Joule's Law**: $P = I^2 R = \frac{V^2}{R} = V \cdot I$.

Resistor Types, Color Coding & Tolerance

- **Resistor Construction Types:**
 - **Carbon Film & Metal Film Resistors:** Thin resistive film spiraled around a ceramic rod, providing high precision (**1%** tolerance) and low thermal noise.
 - **Wirewound Resistors:** Nichrome wire coiled around a ceramic core, capable of handling high power dissipation (**5W to 50W**) in power supply bleeder circuits.
 - **Surface Mount Device (SMD) Resistors:** Rectangular ceramic chips (e.g., 0402, 0603, 0805 packages) with three-digit or four-digit EIA coding soldered directly onto motherboard PCB traces.
- **Standard 4-Band Resistor Color Code:**

$$R = (10 \cdot \text{Band 1} + \text{Band 2}) \times 10^{\text{Multiplier}} \pm \text{Tolerance } \%$$

Colors: Black (0), Brown (1), Red (2), Orange (3), Yellow (4), Green (5), Blue (6), Violet (7), Gray (8), White (9). Tolerance: Gold ($\pm 5\%$), Silver ($\pm 10\%$).

Circuit Laws & Functions in Computer Hardware:

- **Kirchhoff's Current Law (KCL):** The algebraic sum of currents entering any circuit node equals zero: $\sum I_{in} = \sum I_{out}$. Essential for analyzing current distribution across parallel motherboard power rails.
- **Kirchhoff's Voltage Law (KVL):** The algebraic sum of all potential differences around any closed loop in a circuit equals zero: $\sum V = 0$. Used in voltage regulator feedback loop calculations.
- **Current Limiting:** Protecting sensitive LEDs, optocouplers, and transistor base junctions from over-current destruction ($R = \frac{V_{cc} - V_D}{I_D}$).
- **Voltage Division:** Scaling down motherboard voltage rails using series resistors:

$$V_{out} = V_{in} \cdot \left(\frac{R_2}{R_1 + R_2} \right)$$

- **Pull-Up and Pull-Down Resistors:** In digital logic buses (I^2C , PCIe SMBus, GPIO), an unconnected input pin will "float" between logic 0 and 1 due to electromagnetic noise. A **pull-up resistor** (**4.7 k Ω to 10 k Ω**) connects the line to V_{CC} to establish a default high state (1), while a **pull-down resistor** connects the line to Ground (0).
- **Bus Termination:** Matching transmission line characteristic impedance (**50 Ω**) on high-speed DDR memory and PCIe traces to prevent signal reflections and jitter.

7.2 Capacitors

A **Capacitor** is a passive component capable of storing electrical potential energy in an electrostatic field formed between two conductive parallel plates separated by an insulating **dielectric** medium.

$$Q = C \cdot V \implies C = \frac{Q}{V} = \frac{\epsilon_r \epsilon_0 A}{d}$$

where ϵ_0 is the permittivity of free space ($8.854 \times 10^{-12} \text{ F/m}$), ϵ_r is the relative dielectric constant, A is plate area, and d is plate separation distance.

The energy stored in the electrostatic field is $E = \frac{1}{2} CV^2$.

In AC circuits, a capacitor presents **Capacitive Reactance** (X_C), measured in Ohms (Ω):

$$X_C = \frac{1}{2\pi fC}$$

At DC ($f = 0\text{ Hz}$), $X_C = \infty$ (blocks DC current). At ultra-high frequencies ($f \rightarrow \infty$), $X_C \rightarrow 0$ (acts as a low-impedance short circuit to ground for AC noise).

The transient charging and discharging of a capacitor across a series resistor is governed by the exponential equation with time constant $\tau = R \cdot C$:

$$V(t) = V_{max} \left(1 - e^{-\frac{t}{RC}}\right)$$

Capacitor Technologies & Hardware Roles:

- **Multi-Layer Ceramic Capacitors (MLCC):** Miniature surface-mount capacitors with low Equivalent Series Resistance (ESR) and low inductance. Placed by the hundreds on the underside of CPU/GPU sockets for **High-Frequency Decoupling and Power Plane Bypassing**.
- **Solid Polymer Aluminum Electrolytic Capacitors:** High-capacitance (**100µF to 1500µF**) capacitors with conductive polymer electrolytes, engineered to filter low-frequency voltage ripple on motherboard VRMs and prevent bulges or electrolyte drying.
- **DRAM Memory Cells:** In dynamic RAM, every single binary bit is stored as a minute charge (**30 femtofarads**) in a microscopic 3D trench capacitor.

7.3 Inductors

An **Inductor** (choke/coil) is a passive component consisting of a conductive insulated wire coiled around a magnetic core (ferrite or powdered iron), storing energy in a dynamic **magnetic field**.

Governed by **Faraday's Law of Induction** and **Lenz's Law**, any change in current induces an opposing back-EMF:

$$V_L = L \cdot \frac{dI}{dt}$$

The inductive reactance is $X_L = 2\pi fL$. An inductor allows pure DC ($f = 0$) to pass with zero reactive impedance, while opposing and choking high-frequency AC ripples. The energy stored in the magnetic field is $E = \frac{1}{2}LI^2$.

Functions in Computer Hardware:

- **Voltage Regulator Modules (VRMs):** High-current ferrite-core power chokes work alongside high-side and low-side switching MOSFETs in multi-phase buck-converter topologies to step down **+12V** SMPS power to ultra-stable **+1.1V** CPU core voltage with minimal ripple.
- **Electromagnetic Interference (EMI) Suppression:** Toroidal ferrite chokes on USB and power cables filter high-frequency common-mode electromagnetic noise.
- **Ferrite Beads:** Small SMD passive lossy beads that act as high-frequency resistors to attenuate RF noise on analog audio and clock traces.

Unit 8: Motherboard Components — Core Subsystems

The **Motherboard** (system board / mainboard) is the central multi-layer Printed Circuit Board (PCB) — typically containing 6 to 12 copper layers — that serves as the physical and electrical backbone of a computer system, interconnecting the processor, memory, storage, expansion cards, and peripheral interfaces.

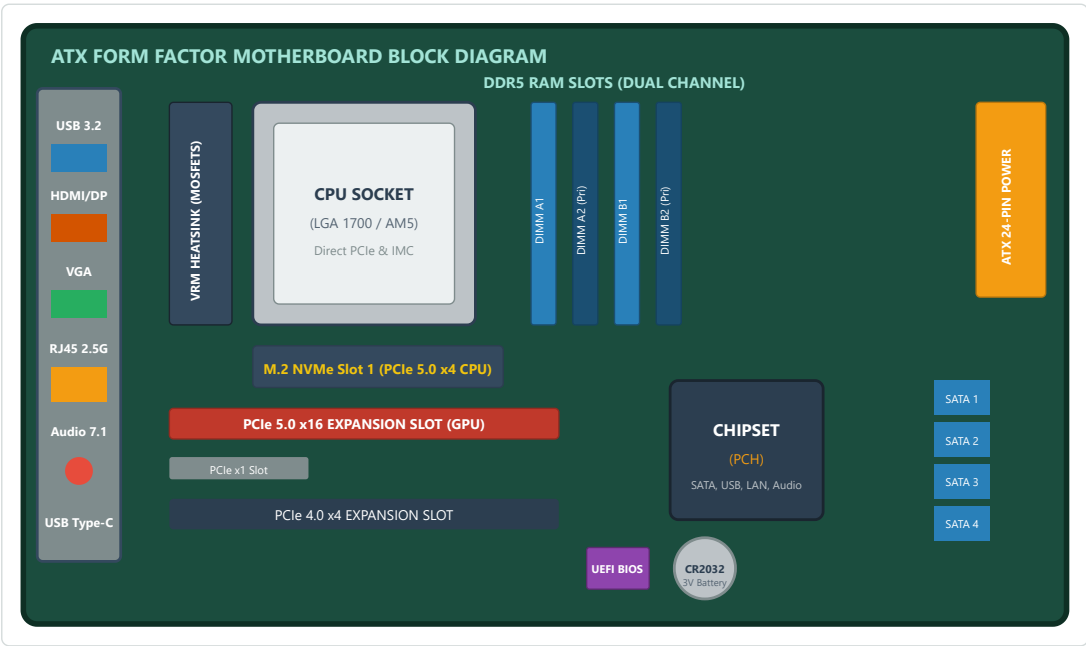


Figure 2.2: Comprehensive Architecture of Modern Motherboard Subsystems, Sockets, Slots, and Interconnects.

8.1 Form Factors & Mechanical Dimensions

- **Standard ATX (Advanced Technology eXtended):** 305 mm × 244 mm (12.0 in × 9.6 in). Up to 7 expansion slots, 4 DIMM slots, maximum VRM cooling phases.
- **Micro-ATX (mATX):** 244 mm × 244 mm (9.6 in × 9.6 in). Up to 4 expansion slots.
- **Mini-ITX:** 170 mm × 170 mm (6.7 in × 6.7 in). Compact form factor for small home-theater and portable PC builds, limited to 1 PCIe slot and 2 DIMM slots.

8.2 CPU Socket, VRM Power Stages & Cooling

- **Socket Architecture:**
 - **Land Grid Array (LGA):** Used primarily by Intel (e.g., LGA 1700, LGA 1851) and modern AMD (AM5). The delicate conductive pins are mounted inside the motherboard socket, while the underside of the CPU features flat gold contact pads. Reduces risk of damaging CPU pins during transit.
 - **Pin Grid Array (PGA):** Legacy architecture (e.g., AMD AM4) where rigid gold-plated pins project from the underside of the CPU package and insert into a Zero Insertion Force (ZIF) socket.
 - **Ball Grid Array (BGA):** Permanent surface-mount solder balls used in laptops, smartphones, and embedded SoC systems. Non-upgradable.
- **Voltage Regulator Module (VRM) Power Stages:** Modern CPUs consume **100A to 300A** at **~ 1.1V**. Motherboards deploy multi-phase synchronous buck converters (e.g., 16+2+1 power phases using integrated Smart Power Stages / DrMOS). A PWM controller interleaves the phases with phase offsets ($\theta = \frac{360^\circ}{N}$), reducing current ripple and thermal stress.
- **Thermal Management & Cooling Solutions:**
 - **Thermal Design Power (TDP):** The maximum theoretical heat energy (in Watts) generated by the CPU under maximum real-world workload that the cooling system is engineered to dissipate.
 - **Thermal Interface Material (TIM):** Microscopic imperfections on the CPU Integrated Heat Spreader (IHS) and heatsink base trap air pockets (air is a poor thermal conductor: $k \approx 0.026 \text{ W/m} \cdot \text{K}$). Thermal paste (silicone suspended with zinc oxide, aluminum, or silver particles) fills these voids to maximize conductive heat transfer ($k \approx 5\text{--}12 \text{ W/m} \cdot \text{K}$).
 - **Active Air Cooling:** Copper heat pipes containing a low-boiling-point working fluid (evaporating at the CPU base, rising as vapor to aluminum fins, condensing, and returning via capillary wick action) paired with a 4-pin PWM (Pulse Width Modulation) controlled fan.
 - **All-In-One (AIO) Liquid Cooling:** Closed-loop system circulating liquid coolant via a mechanical water pump from a copper water-block across a multi-channel radiator cooled by static-pressure fans.

8.3 RAM Slots (DIMM) & Memory Channels

- **Form Factors:** Dual In-line Memory Modules (DIMMs, 288 pins on DDR4/DDR5) for desktops; Small Outline DIMMs (SO-DIMMs, 260/262 pins) for laptops.
- **Multi-Channel Architecture:**
 - **Single-Channel:** One 64-bit wide data bus between CPU memory controller and RAM.
 - **Dual-Channel:** Two independent 64-bit memory buses (**128-bit** aggregate width), doubling theoretical memory bandwidth. Requires population of matched pairs in alternating slots (typically slots A2 and B2).
 - **DDR5 Sub-channels:** DDR5 divides each DIMM into two independent 32-bit channels (plus 8-bit ECC), improving access efficiency.

8.4 Expansion Slots (PCI Express — PCIe)

PCIe is a high-speed, serial, point-to-point interconnect standard replacing older parallel PCI and AGP buses.

- **Differential Signaling Lanes:** Each PCIe lane (**x1**) consists of two pairs of differential copper traces (one pair for transmitting, one pair for receiving full-duplex). Slots are sized physically and electrically as **x1, x4, x8, and x16**.
- **PCIe Protocol Stack:**
 - *Transaction Layer:* Generates Transaction Layer Packets (TLP) for memory reads/writes and I/O.
 - *Data Link Layer:* Handles error detection (LCRC), ACK/NAK flow control, and retransmission.
 - *Physical Layer:* Handles 8b/10b (Gen 1/2) or 128b/130b (Gen 3/4/5) serialization and clock recovery.
- **PCIe Generations Bandwidth:**
 - **PCIe 3.0:** 8 GT/s (1 GB/s per lane simplex $\Rightarrow$ **16 GB/s on x16**). Uses 128b/130b encoding (**98.46%** efficiency).
 - **PCIe 4.0:** 16 GT/s (2 GB/s per lane $\Rightarrow$ **32 GB/s on x16**).
 - **PCIe 5.0:** 32 GT/s (4 GB/s per lane $\Rightarrow$ **64 GB/s on x16**).

8.5 Chipset Architecture: Northbridge/Southbridge to Modern PCH

- **Legacy Architecture:**
 - **Northbridge (Memory Controller Hub - MCH):** Connected directly to CPU via Front Side Bus (FSB), handling high-speed subsystems: System RAM and AGP/PCIe graphics.
 - **Southbridge (I/O Controller Hub - ICH):** Connected to Northbridge, managing slower peripherals: PCI slots, SATA, IDE, USB, BIOS, and Audio.
 - **Modern Platform Controller Hub (PCH) Architecture:** Modern CPUs integrate the Memory Controller (IMC) and primary PCIe **x16** GPU and M.2 lanes directly on the CPU die. The motherboard contains a single **PCH chipset** connected to the CPU via a high-speed proprietary link (e.g., Intel DMI 4.0 or AMD PCIe uplink), managing secondary storage, USB controllers, SATA ports, Wi-Fi, and Gigabit Ethernet.
-

Unit 9: Motherboard Components — Firmware, Storage & I/O Interfaces

9.1 BIOS / UEFI Firmware Subsystem

The **Firmware** is low-level non-volatile system software permanently stored on an onboard SPI Flash EEPROM chip on the motherboard.

- **Legacy BIOS (Basic Input/Output System):** Operates in 16-bit x86 Real Mode, constrained to 1MB of memory addressability. Relies on software interrupts (e.g., INT 13h) and the Master Boot Record (MBR).
- **Modern UEFI (Unified Extensible Firmware Interface):** 32-bit or 64-bit execution environment supporting rich graphical user interfaces, mouse input, direct network stack boot (PXE), and drive capacities exceeding **2.2 TB**.
 - **Secure Boot:** Cryptographic validation mechanism where the UEFI firmware verifies the digital signature of the OS bootloader against public keys embedded in NVRAM (Platform Key PK, Key Exchange Key KEK, Allowed Signature Database db, Forbidden Database dbx), protecting against low-level rootkits.
 - **ACPI (Advanced Configuration and Power Interface):** Power management standard defining processor power states (C0 active to C10 deep sleep) and system sleep states (S0 working, S3 suspend to RAM, S4 hibernation, S5 soft power off).
 - **CMOS Battery:** A lithium coin-cell battery (typically **CR2032, 3V**) powering the motherboard's Real-Time Clock (RTC) oscillator and maintaining non-volatile CMOS configuration registers when AC power is disconnected.

9.2 Storage Interfaces: SATA vs M.2 NVMe

- **Serial ATA (SATA III):** Operates at **6.0 Gbps** (**550 MB/s** net throughput) using the legacy **AHCI (Advanced Host Controller Interface)** protocol designed for spinning disks, limited to a single command queue of 32 commands with Native Command Queuing (NCQ).
- **M.2 NVMe (Non-Volatile Memory Express):** Compact form-factor drives (standard **22mm × 80mm**, form-factor 2280) connecting directly to PCIe lanes. Uses the lightweight **NVMe protocol**, supporting up to **64,000** parallel submission and completion queues each with **64,000** commands, unlocking speeds of **3,500 MB/s** (Gen 3), **7,500 MB/s** (Gen 4), and **14,000 + MB/s** (Gen 5).

9.3 External Display & Peripheral Ports

- **Video Interfaces:**
 - **VGA (Video Graphics Array):** 15-pin D-sub analog RGB interface. Subject to analog signal attenuation and noise at high resolutions.
 - **HDMI (High-Definition Multimedia Interface):** Digital audio/video interface using TMDS (Transition-Minimized Differential Signaling). HDMI 2.1 supports up to **48 Gbps** (4K @ 120Hz, 8K @ 60Hz, Dynamic HDR).
 - **DisplayPort (DP):** Digital packetized protocol similar to Ethernet; supports Daisy-Chaining via Multi-Stream Transport (MST).
- **Universal Serial Bus (USB):**
 - **USB 2.0: 480 Mbps** (4-pin connector: V_{BUS} , $D-$, $D+$, GND).
 - **USB 3.2 Gen 1 (USB 3.0): 5 Gbps** (SuperSpeed, 9 pins).
 - **USB 3.2 Gen 2x2: 20 Gbps** dual-lane over Type-C.
 - **USB4 / Thunderbolt 4: 40 Gbps**, integrating PCIe tunneling, DisplayPort Alternate Mode, and USB Power Delivery (USB-PD up to 240W, 48V @ 5A). Reversible 24-pin Type-C connector.
- **Network Interface (RJ-45 Ethernet):** 8-pin 8P8C modular connector interfacing onboard PHY transceiver with unshielded twisted pair (UTP Cat 5e/6/6a) cabling, supporting **1 Gbps**, **2.5 Gbps**, or **10 Gbps** full-duplex Ethernet using CSMA/CD and pulse-amplitude modulation.

Unit 10: Computer Components — Power Supply & Storage Devices

10.1 Switched-Mode Power Supply (SMPS)

The **SMPS** is an electronic power supply unit that converts high-voltage alternating current (AC mains: **110V/230V, 50/60 Hz**) into smooth, regulated low-voltage direct current (DC) rails required by internal computer hardware.

Detailed Operational Stages of an SMPS:

1. **Input EMI Filtering:** Inductive-capacitive network suppresses incoming line spikes and prevents high-frequency switching noise from feeding back into the electrical grid.
2. **Primary Rectification & Smoothing:** Bridge rectifier diodes convert AC to high-voltage DC, smoothed by large electrolytic reservoir capacitors ($\approx$ **320V DC**).
3. **High-Frequency Inversion (Switching):** High-power MOSFET transistors switch the high-voltage DC ON and OFF at ultra-high frequencies (**50 kHz to 500 kHz**) using Pulse Width Modulation (PWM).
4. **Step-Down High-Frequency Transformer:** Because transformer core size is inversely proportional to operating frequency ($\text{Area} \propto \frac{1}{f}$), high frequency allows miniature, highly efficient ferrite transformers to step down voltages.
5. **Secondary Rectification & Filtering:** Schottky diodes rectify high-frequency pulses back to DC, smoothed by inductors and solid capacitors into clean output rails.
6. **Feedback & Supervisory Control:** Optocouplers provide galvanic isolation while monitoring output rails, adjusting PWM duty cycles dynamically, and asserting the **Power Good** (**PWR_OK** / **+5V**) signal to the CPU only after voltages stabilize.

80 PLUS Energy Efficiency Certifications (at 50% Load)

Certification Tier	Efficiency at 20% Load	Efficiency at 50% Load	Efficiency at 100% Load
80 PLUS Standard	80%	80%	80%
80 PLUS Bronze	82%	85%	82%
80 PLUS Silver	85%	88%	85%
80 PLUS Gold	87%	90%	87%
80 PLUS Platinum	90%	92%	89%
80 PLUS Titanium	92%	94%	90%

Standard ATX Voltage Rails

Voltage Rail	Wire Color (ATX Standard)	Primary Hardware Consumers
+12V	Yellow	CPU VRMs, GPU PCIe power, cooling fans, spindle motors
+5V	Red	Motherboard logic circuits, SATA SSD/HDD logic, USB peripheral power
+3.3V	Orange	RAM slots, PCIe slot logic, Chipset, M.2 NVMe drives
+5VSB	Purple	Standby power (powers power button, Wake-on-LAN, RTC when PC is OFF)
PS\ON\#	Green	Active-low signal grounded by motherboard to turn on main DC rails

10.2 Storage Devices: HDD vs SATA SSD vs NVMe SSD

- **Hard Disk Drive (HDD):** Electromechanical magnetic storage. Data is recorded on spinning ferromagnetic aluminum/glass platters (**5400 or 7200 RPM**) divided into concentric *tracks*, *cylinders*, and *sectors* (**512e** or **4Kn** bytes). Read/write heads fly on an aerodynamic air bearing $\approx 10\text{ nm}$ above platters on a voice-coil actuator arm.
 - **Performance Metrics:**

$$\text{Average Access Time} = \text{Average Seek Time} + \text{Average Rotational Latency} + \text{Transfer Time}$$

Mechanical Seek Time (**4–10 ms**) and Rotational Latency (**4.16 ms** at 7200 RPM) constrain random I/O performance to $\approx 75\text{--}150\text{ IOPS}$.
 - **Recording Technologies:** Conventional Perpendicular Magnetic Recording (PMR/CMR) vs Shingled Magnetic Recording (SMR - overlapping tracks for high capacity at the expense of random write speed).
- **Solid State Drive (SSD):** Pure solid-state non-volatile semiconductor storage utilizing **NAND Flash Memory** (floating-gate or charge-trap transistors storing electrons).
 - **NAND Flash Hierarchy:** Memory is organized into *Dies* $\rightarrow$ *Planes* $\rightarrow$ *Blocks* (erase unit, **2–8 MB**) $\rightarrow$ *Pages* (read/write unit, **4–16 KB**). Crucially, NAND flash **cannot overwrite existing data in place**; a page must be erased (at the block level) before writing.
 - **NAND Cell Types:**
 - **SLC (Single-Level Cell):** 1 bit/cell (**2** voltage levels). $\sim 100,000$ write cycles endurance, highest speed.
 - **MLC (Multi-Level Cell):** 2 bits/cell (**4** voltage levels). $\sim 10,000$ cycles.
 - **TLC (Triple-Level Cell):** 3 bits/cell (**8** voltage levels). $\sim 3,000$ cycles. Dominant consumer standard.
 - **QLC (Quad-Level Cell):** 4 bits/cell (**16** voltage levels). $\sim 1,000$ cycles. High density, budget storage.
 - **Flash Management:** The SSD controller executes a **Flash Translation Layer (FTL)** handling *Wear Leveling* (distributing write/erase cycles evenly across blocks), **SLC Caching**, and **TRIM** command support (allowing OS to notify SSD when deleted file blocks can be proactively garbage-collected, preventing Write Amplification Factor — WAF degradation).

Unit 11: Memory Subsystem — RAM, ROM & Cache Hierarchy

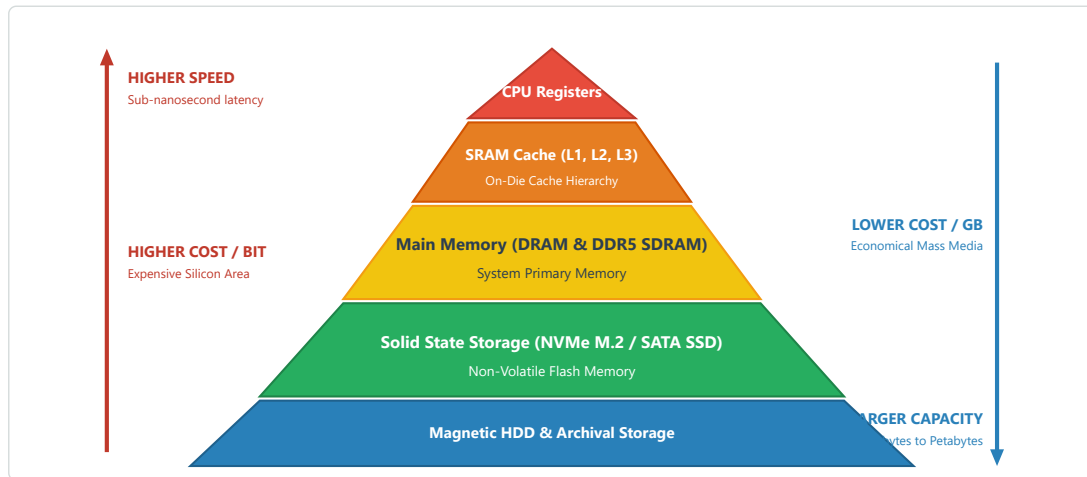


Figure 2.3: The Computer Storage and Memory Hierarchy displaying Latency, Capacity, and Cost Trade-offs.

11.1 Random Access Memory (RAM)

RAM is high-speed, volatile read-write semiconductor primary memory directly addressable by the CPU.

1. Dynamic RAM (DRAM) vs Static RAM (SRAM)

- **DRAM (1T-1C Cell):** Each bit is stored as an electrical charge on a single microscopic capacitor accessed via a single MOSFET pass-transistor. Because dielectric charge leaks over milliseconds, DRAM requires constant **periodic refresh cycles** (re-reading and rewriting every row every **64 ms**). High density and low cost make DRAM the standard for main memory.
- **SRAM (6T Bistable Latch):** Each bit is stored in a flip-flop circuit comprising six cross-coupled CMOS transistors. As long as power is supplied, SRAM holds its data indefinitely **without refresh cycles**. Access latencies are sub-nanosecond (**0.5–2 ns**), making SRAM the universal choice for CPU L1/L2/L3 on-die caches, though its larger physical size (**6×** silicon area of DRAM) makes it expensive.

2. Evolution of Synchronous Dynamic RAM (SDRAM & DDR)

SDRAM synchronizes all memory transfers to the system memory bus clock. **Double Data Rate (DDR)** transmits data on both the *rising edge* and *falling edge* of the clock signal, doubling theoretical throughput.

- **Prefetch Architecture:** DDR (**2n** prefetch), DDR2 (**4n**), DDR3 (**8n**), DDR4 (**8n**), DDR5 (**16n** prefetch).
- **DDR5 Key Advancements:** Integrated on-die Error-Correcting Code (ECC), dual independent 32-bit channels per module, on-module Power Management IC (PMIC) shifting voltage regulation from the motherboard to the DIMM, and operating voltage reduced to **1.1V**.
- **Memory Timings & CAS Latency:** Expressed as **CL-tRCD-tRP-tRAS**. **CAS Latency (CL)** is the exact clock cycle delay between the memory controller asserting a Column Address Strobe and data being placed on the bus.

$$\text{True Latency (ns)} = \frac{\text{CAS Latency} \times 2000}{\text{Data Rate (MT/s)}}$$

11.2 Read-Only Memory (ROM)

ROM is non-volatile primary memory that permanently retains instructions and data even when system power is completely disconnected.

- **Mask ROM:** Data permanently hardwired during factory semiconductor lithography. Completely unalterable.
- **PROM (Programmable ROM):** Manufactured blank with microscopic fusible links blown permanently using high electrical programming voltage. One-time programmable (OTP).
- **EPROM (Erasable PROM):** Uses floating-gate transistors. Erased by exposing the quartz glass window atop the IC package to intense Ultraviolet (UV) light for 20–30 minutes.
- **EEPROM (Electrically Erasable PROM):** Erased and reprogrammed byte-by-byte electrically using Fowler-Nordheim electron tunneling, without removal from the circuit board.
- **Flash Memory:** High-density block-erasable EEPROM. Modern UEFI BIOS chips use SPI Flash ROM.

11.3 Cache Memory Subsystem & Mapping Techniques

Cache Memory is ultra-fast SRAM placed between the CPU registers and main memory to bridge the processing-speed gap.

1. Principle of Locality of Reference

- **Temporal Locality:** If a memory location is accessed once, it is highly likely to be accessed again in the near future (e.g., loop counters, subroutine code).
- **Spatial Locality:** If a specific memory word is accessed, adjacent memory locations are highly likely to be accessed soon (e.g., sequential instruction execution, contiguous array traversal).

2. Cache Mapping Techniques & Address Partitioning

A physical memory address of m bits is partitioned into three fields: **Tag | Index / Set | Word Offset**.

1. **Direct Mapping:** Each main memory block maps to exactly one predetermined cache line:

$$\text{Cache Line} = (\text{Block Address}) \pmod{\text{Total Cache Lines}}$$

$$\text{Offset Bits} = \log_2(\text{Block Size}), \quad \text{Index Bits} = \log_2(\text{Number of Lines}), \quad \text{Tag Bits} = m - (\text{Index} + \text{Offset})$$

Advantage: Fast, simple tag comparison. *Disadvantage:* High conflict misses if two active memory blocks map to the same line.

2. **Fully Associative Mapping:** A main memory block can be loaded into *any* available cache line.

$$\text{Offset Bits} = \log_2(\text{Block Size}), \quad \text{Tag Bits} = m - \text{Offset}$$

Advantage: Zero conflict misses; maximum hit rate. *Disadvantage:* Requires expensive associative hardware comparators to search all cache tags in parallel.

3. **N -Way Set-Associative Mapping:** The cache is partitioned into S sets, each containing N cache lines ($S = \frac{\text{Total Lines}}{N}$). A memory block maps to a specific set based on index bits, but can occupy any of the N lines within that set.

$$\text{Set Index Bits} = \log_2(S), \quad \text{Tag Bits} = m - (\text{Set Index} + \text{Offset})$$

Represents the optimal engineering compromise in modern CPU L1/L2/L3 caches (e.g., 8-way or 16-way set associativity).

WORKED EXAMPLE 11.1: CACHE ADDRESS PARTITIONING

Problem: A 32-bit byte-addressable system has a **64 KB** 4-way set-associative cache with **64-byte** cache blocks. Calculate the bit-widths of the Word Offset, Set Index, and Tag fields.

Solution:

- Block Size = **64 bytes** = 2^6 bytes $\implies$ **Offset Bits = 6 bits**.
- Total Cache Size = **64 KB** = **65,536 bytes**.
- Total Lines in Cache = $\frac{65,536}{64} = 1,024$ lines.
- Number of Sets $S = \frac{\text{Total Lines}}{N} = \frac{1024}{4} = 256$ sets = 2^8 sets $\implies$ **Set Index Bits = 8 bits**.
- Tag Bits = $32 - (\text{Set Index} + \text{Offset}) = 32 - (8 + 6) = 18$ bits.

The 32-bit address is partitioned as: **[Tag : 18 bits] | [Set : 8 bits] | [Offset : 6 bits]**.

3. Cache Write Policies & Replacement Algorithms

- **Write Policies:**
 - **Write-Through:** Every write to the cache is simultaneously written to main memory. High bus traffic, but memory is always completely synchronized.
 - **Write-Back:** Writes are made only to the cache line, marking a "Dirty Bit". The modified line is written back to main DRAM only when it is evicted by a replacement algorithm. Maximizes performance by minimizing memory bus writes.
- **Cache Replacement Algorithms:**
 - **Least Recently Used (LRU):** Evicts the cache block that has gone unreferenced for the longest duration (highest hit rate, requires hardware timestamp/counter tracking).
 - **First-In First-Out (FIFO):** Evicts the oldest loaded block regardless of access recency.
 - **Pseudo-LRU / Random:** Low-overhead approximations for high-associativity caches (**16-way** to **64-way**).

Fundamentals of Computers and Computational Thinking

Course Code: CS101 Credits: 4 (Lecture: 3, Practical: 2) Total Hours: 75 Hrs

MODULE III: Software Systems, Operating Systems & Device Drivers

Comprehensive Academic Notes — Units 12 through 15 (Duration: 10 Hours, Weightage: 15 Marks)

Unit 12: Softwares — Classification, Taxonomy & Translators

CONCEPTUAL DEFINITION: SOFTWARE

Software is an organized collection of computer programs, algorithmic procedures, data structures, and associated technical documentation that directs computer hardware on precisely how to execute data acquisition, processing, manipulation, transmission, and storage tasks.

While hardware represents the physical semiconductor circuitry and mechanical packaging, software represents the logical instructions that animate the machine. Software is classified into two overarching domains: **System Software** and **Application Software**.

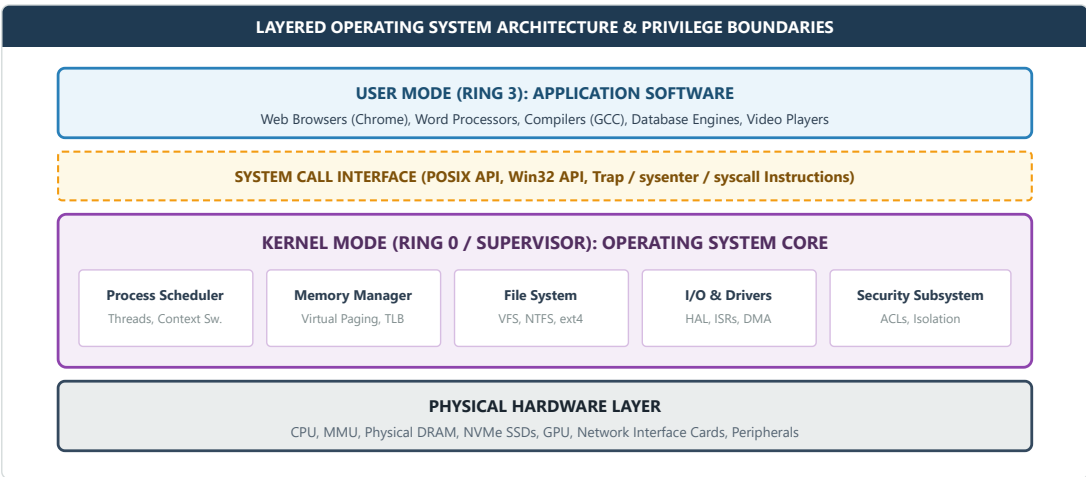


Figure 3.1: Layered Architecture of Computer Software and Operating System Kernel Subsystems.

12.1 System Software

System Software comprises low-level programs that directly manage, control, and interact with the physical computer hardware, providing a stable, abstracted platform upon which application software executes.

1. Operating Systems (OS)

The primary supervisory control program managing hardware resources and presenting an abstract virtual machine interface (e.g., Windows 11, Ubuntu Linux, macOS, FreeBSD).

2. Programming Language Generations & Taxonomy

Programming languages have evolved through five distinct generations (1GL to 5GL), spanning multiple computational paradigms:

- First-Generation Language (1GL):** Pure binary machine code (**0s and 1s**) directly executed by CPU instruction decoders. Extremely tedious, non-portable, and prone to human error.
- Second-Generation Language (2GL):** Assembly languages using symbolic mnemonics (e.g., `ADD`, `MOV`, `JMP`, `PUSH`) translated 1-to-1 to binary machine code by an **Assembler**.
- Third-Generation Language (3GL):** High-level imperative procedural and object-oriented languages (C, C++, Fortran, Java, Pascal) using English keywords and mathematical expressions, translated via Compilers or Interpreters.
- Fourth-Generation Language (4GL):** Non-procedural, declarative query and data manipulation languages focusing on *what* to achieve rather than *how* to compute (e.g., SQL, MATLAB, SAS).
- Fifth-Generation Language (5GL):** Constraint-based and logic programming languages used in artificial intelligence and automated theorem proving (e.g., Prolog, Mercury).

3. Programming Paradigms and Typing Systems

- Imperative / Procedural Paradigm:** Programs are organized around state variables and step-by-step sequences of imperative statements (C, Pascal).

- **Object-Oriented Paradigm (OOP):** Programs model domain entities as encapsulated objects containing private state attributes and public methods (Java, C++, C#). Enforces Encapsulation, Inheritance, and Polymorphism.
- **Functional Paradigm:** Computation is treated as the mathematical evaluation of pure, immutable functions without mutable side effects (Haskell, Lisp, Scala, Erlang).
- **Type Systems:**
 - *Static Typing vs Dynamic Typing:* In statically typed languages (C, Rust, Java), variable data types are checked and verified at compile-time; in dynamically typed languages (Python, JavaScript, Ruby), types are associated with runtime values and checked during execution.
 - *Strong Typing vs Weak Typing:* Strongly typed languages strictly disallow implicit invalid type coercion (e.g., adding an integer to a string raises a runtime error), whereas weakly typed languages perform implicit casting (e.g., C pointer casting, JavaScript string-number concatenation).

4. Memory Management & Garbage Collection

- **Stack Memory vs Heap Memory:**
 - *Stack:* LIFO (Last-In First-Out) memory segment automatically managed by CPU stack pointer register (`RSP`). Stores local variables, return addresses, and function call frames. Extremely fast ($O(1)$ allocation/deallocation), but fixed in size.
 - *Heap:* Dynamically allocated memory pool managed manually (`malloc()` / `free()` in C, `new` / `delete` in C++) or automatically via a runtime Garbage Collector (GC). Susceptible to memory fragmentation, buffer overflows, dangling pointers, and memory leaks.
- **Automatic Garbage Collection Strategies:**
 - *Reference Counting:* Tracks active references to every allocated object; frees memory when count drops to zero. *Limitation:* Fails to reclaim cyclic circular references unless augmented with cycle detection.
 - *Tracing Garbage Collection (Mark-and-Sweep):* Traverses the graph of live object references starting from root pointers (stack frames, global registers). The *Mark phase* sets a bit on reachable objects; the *Sweep phase* traverses the entire heap, reclaiming memory occupied by unmarked unreachable objects.
 - *Generational GC:* Exploits the weak generational hypothesis (most objects die shortly after allocation). The heap is partitioned into *Young Generation (Eden + Survivor spaces)* and *Old / Tenured Generation*. Frequent minor collections scavenge the Eden space with minimal pause latency.

5. Language Translators: Compilers, Interpreters & Assemblers

- **Assembler:** Translates assembly mnemonics into object code, resolving symbolic labels into relative memory offsets via two passes (Pass 1 builds Symbol Table, Pass 2 emits machine code).
- **Compiler:** Translates entire high-level source code programs into target machine code or intermediate bytecode in a single batch process before execution. Generates optimized standalone binaries (e.g., GCC, Clang, MSVC).
- **Interpreter:** Reads, parses, and executes high-level source code line-by-line in real time without creating standalone machine binaries (e.g., CPython, standard Ruby). Provides rapid development and dynamic typing at the cost of slower execution speeds.
- **Hybrid / Just-In-Time (JIT) Compilers:** Modern runtimes (Java Virtual Machine JVM, V8 JavaScript engine, .NET CLR) first compile source code into portable intermediate Bytecode, and then dynamically compile hot code loops into native CPU machine instructions at runtime using JIT compilation.

6. Deep Dive: Six Phases of a Modern Compiler

1. **Lexical Analysis (Scanner):** Scans the input character stream and groups them into lexical units called **Tokens** (e.g., `<KEYWORD, "while">`, `<IDENTIFIER, "counter">`, `<REL_OP, "<=">`, `<NUM, 100>`), while discarding whitespace and comments.
2. **Syntax Analysis (Parser):** Implements formal Context-Free Grammars (CFG) using algorithms like LL(k), LR(k), or LALR to verify grammatical correctness and construct an **Abstract Syntax Tree (AST)**.
3. **Semantic Analysis:** Verifies type safety, enforces variable scope and declaration rules, checks array bounds constraints, and ensures function parameter compatibility using the Symbol Table.
4. **Intermediate Code Generation (ICG):** Translates the validated AST into a machine-independent intermediate language, typically **Three-Address Code (TAC)**:

$$t_1 = y \times z, \quad t_2 = x + t_1$$

5. **Code Optimization:** Transforms intermediate code to improve execution speed and reduce memory consumption:
 - *Constant Folding & Propagation:* Pre-calculating compile-time constants (e.g., replacing `3 + 5` with `8`).
 - *Dead Code Elimination:* Removing code segments that can never execute or whose results are never used.
 - *Loop-Invariant Code Motion:* Moving computations outside loops if their results do not change across iterations.
 - *Strength Reduction:* Replacing computationally expensive operations with cheaper ones (e.g., replacing `x × 2` with `x << 1`).
 - *Common Subexpression Elimination:* Identifying redundant calculations and caching their results.
6. **Target Code Generation:** Maps intermediate variables to physical CPU registers using Graph Coloring Algorithms and emits native binary machine code instructions.

7. Linkers, Loaders & Binary Formats

- **Static Linking vs Dynamic Linking:**
 - *Static Linker:* Copies all required library routines (e.g., `libc.a`) directly into the final executable binary, resulting in larger files that run independently without external dependency issues.
 - *Dynamic Linker:* Inserts symbolic references to shared libraries (Dynamic Link Libraries — `.dll` on Windows, Shared Objects — `.so` on Linux). Multiple executing programs share a single copy of library code in RAM, reducing memory footprint. Relocation is handled at runtime via the **Global Offset Table (GOT)** and **Procedure Linkage Table (PLT)** for Position-Independent Code (PIC).
- **Executable File Formats (ELF & PE):**

- **Executable and Linkable Format (ELF - Linux):** Contains an ELF Header, Program Header Table (segments mapped to memory), Section Header Table (`.text` code, `.data` initialized globals, `.bss` zero-initialized globals, `.rodata` read-only constants, `.symtab` symbol table), and dynamic linking tables.
- **Portable Executable (PE - Windows):** Extends the MS-DOS MZ header with PE Header, Optional Header, Data Directories (Export/Import tables), and section headers (`.text` , `.rdata` , `.data` , `.pdata`).
- **Loader:** The OS program that reads the executable file header, allocates virtual memory segments (Code/Text, Initialized Data, BSS, Stack, Heap), maps shared libraries, initializes CPU registers, and transfers execution to the program entry point (`main()`).

12.2 Application Software

Application Software encompasses programs engineered to perform end-user tasks, productivity workflows, or business data processing:

- **General-Purpose Software:** Mass-market productivity applications: Word Processors (Microsoft Word, LibreOffice Writer), Spreadsheets (Microsoft Excel), Web Browsers (Google Chrome, Mozilla Firefox), and Media Players (VLC).
- **Specialized / Bespoke Software:** Custom-engineered software tailored to specific domains: Medical PACS DICOM imaging systems, Aircraft Avionics control, Electronic Design Automation (EDA - Cadence, KiCad), and CAD/CAM modeling (AutoCAD, SolidWorks).
- **Enterprise Systems:** Large-scale multi-user business platforms: Enterprise Resource Planning (ERP - SAP S/4HANA), Customer Relationship Management (CRM - Salesforce), and Relational Database Engines (PostgreSQL, Oracle DB).

Comparative Matrix: System Software vs Application Software

Evaluation Parameter	System Software	Application Software
Primary Purpose	Manages, controls, and abstracts physical hardware	Executes specific end-user business or creative tasks
Hardware Access	Direct register-level and physical I/O interaction	Zero direct hardware access (restricted to OS System Calls)
Execution Privilege	Privileged Kernel Mode (Ring 0 / Supervisor)	Unprivileged User Mode (Ring 3)
Operational Dependency	Runs autonomously; provides runtime environment	Completely dependent on an underlying OS to execute
Programming Languages	C, C++, Rust, Assembly	Python, Java, JavaScript, C#, Swift, PHP, Go

Unit 13: Operating Systems — Architecture, Scheduling, POST & Booting

13.1 Fundamental Responsibilities of an Operating System

An **Operating System (OS)** is the master control program that acts as an intermediary between computer hardware and user applications. It fulfills two primary roles:

- **Extended Machine (Virtual Machine):** Provides clean, high-level abstractions (files, sockets, virtual memory, processes) that hide the low-level complexities of raw hardware registers, interrupts, and disk sectors.
- **Resource Allocator & Arbiter:** Manages finite hardware resources (CPU cores, DRAM, storage space, network bandwidth) among competing user processes fairly, efficiently, and securely.

Core Operating System Subsystems:

- Process Management & CPU Scheduling:** Creating, scheduling, synchronizing, and terminating processes and threads.
 - **7-State Process Model:** Processes transition through: *New* → *Ready* → *Running* → *Waiting/Blocked* → *Terminated*, along with *Ready-Suspended* and *Blocked-Suspended* when swapped out to disk secondary storage.
 - **Process Control Block (PCB):** The kernel data structure storing process state, PID, Program Counter, CPU registers, memory limits, open file descriptors, and scheduling priority.
- Process Synchronization & Deadlocks:**
 - **Critical Section Problem:** Ensures mutual exclusion when multiple concurrent processes access shared memory. A valid solution must satisfy: *Mutual Exclusion*, *Progress*, and *Bounded Waiting*.
 - **Synchronization Primitives:** Semaphores (Counting and Binary), Mutex Locks, Condition Variables, and Spinlocks.
 - **Deadlock & Coffman Conditions:** A state where two or more processes are permanently blocked waiting for resources held by each other. Deadlock can occur if and only if all four **Coffman Conditions** hold simultaneously:
 1. *Mutual Exclusion:* Resources cannot be shared simultaneously.
 2. *Hold and Wait:* Processes hold allocated resources while requesting new ones.
 3. *No Preemption:* Resources cannot be forcibly confiscated from a process.
 4. *Circular Wait:* A closed chain of processes exists where each process waits for a resource held by the next.
 - **Deadlock Handling:** Banker's Algorithm for deadlock avoidance, Resource Allocation Graphs (RAG) for detection, and process termination/preemption for recovery.

3. **Memory Management & Virtual Paging:** Providing each process with an isolated, contiguous virtual address space mapped to physical DRAM via multi-level Page Tables and the hardware Memory Management Unit (MMU).
4. **File System & Secondary Storage:** Structuring persistent disk blocks into hierarchical directories, managing metadata inodes, and enforcing access control lists (ACLs).
5. **I/O Subsystem:** Providing uniform driver interfaces, buffering, caching, spooling, and DMA coordination.

WORKED EXAMPLE 13.1: CPU SCHEDULING COMPARISON

Problem: Four processes arrive at time $t = 0$ with burst times: $P_1 = 6 \text{ ms}$, $P_2 = 8 \text{ ms}$, $P_3 = 7 \text{ ms}$, $P_4 = 3 \text{ ms}$. Calculate Average Waiting Time under FCFS and Shortest Job First (SJF).

1. First-Come First-Served (FCFS):

Gantt Chart: | P_1 (0–6) | P_2 (6–14) | P_3 (14–21) | P_4 (21–24) |

- $WT(P_1) = 0 \text{ ms}$, $WT(P_2) = 6 \text{ ms}$, $WT(P_3) = 14 \text{ ms}$, $WT(P_4) = 21 \text{ ms}$
- Average Waiting Time (FCFS): $\frac{0+6+14+21}{4} = \frac{41}{4} = 10.25 \text{ ms}$.

2. Shortest Job First (SJF - Non-preemptive):

Execution Order: $P_4(3\text{ms}) \rightarrow P_1(6\text{ms}) \rightarrow P_3(7\text{ms}) \rightarrow P_2(8\text{ms})$

Gantt Chart: | P_4 (0–3) | P_1 (3–9) | P_3 (9–16) | P_2 (16–24) |

- $WT(P_4) = 0 \text{ ms}$, $WT(P_1) = 3 \text{ ms}$, $WT(P_3) = 9 \text{ ms}$, $WT(P_2) = 16 \text{ ms}$
- Average Waiting Time (SJF): $\frac{0+3+9+16}{4} = \frac{28}{4} = 7.00 \text{ ms}$ (Optimal).

WORKED EXAMPLE 13.2: BANKER'S ALGORITHM SAFETY EVALUATION

Consider 5 processes (P_0, P_1, P_2, P_3, P_4) and 3 resource types ($A = 10, B = 5, C = 7$).

Currently allocated resources:

$P_0 : (0, 1, 0), \quad P_1 : (2, 0, 0), \quad P_2 : (3, 0, 2), \quad P_3 : (2, 1, 1), \quad P_4 : (0, 0, 2)$

Total Allocated: $A = 7, B = 2, C = 5 \implies \text{Available: } A = 3, B = 3, C = 2$

Maximum Demand Matrix (Max):

$P_0 : (7, 5, 3), \quad P_1 : (3, 2, 2), \quad P_2 : (9, 0, 2), \quad P_3 : (2, 2, 2), \quad P_4 : (4, 3, 3)$

Remaining Need Matrix ($\text{Need} = \text{Max} - \text{Allocation}$):

$P_0 : (7, 4, 3), \quad P_1 : (1, 2, 2), \quad P_2 : (6, 0, 0), \quad P_3 : (0, 1, 1), \quad P_4 : (4, 3, 1)$

Safety Algorithm Execution:

1. Check P_1 : $\text{Need}(1, 2, 2) \leq \text{Available}(3, 3, 2) \implies \text{Execute } P_1$. New Available = $(3, 3, 2) + (2, 0, 0) = (5, 3, 2)$.
2. Check P_3 : $\text{Need}(0, 1, 1) \leq \text{Available}(5, 3, 2) \implies \text{Execute } P_3$. New Available = $(5, 3, 2) + (2, 1, 1) = (7, 4, 3)$.
3. Check P_0 : $\text{Need}(7, 4, 3) \leq \text{Available}(7, 4, 3) \implies \text{Execute } P_0$. New Available = $(7, 4, 3) + (0, 1, 0) = (7, 5, 3)$.
4. Check P_2 : $\text{Need}(6, 0, 0) \leq \text{Available}(7, 5, 3) \implies \text{Execute } P_2$. New Available = $(7, 5, 3) + (3, 0, 2) = (10, 5, 5)$.
5. Check P_4 : $\text{Need}(4, 3, 1) \leq \text{Available}(10, 5, 5) \implies \text{Execute } P_4$. New Available = $(10, 5, 5) + (0, 0, 2) = (10, 5, 7)$.

Conclusion: The system is in a provably **Safe State** with safe sequence $\langle P_1, P_3, P_0, P_2, P_4 \rangle$. No deadlock can occur.

WORKED EXAMPLE 13.3: VIRTUAL MEMORY PAGE REPLACEMENT COMPARISON

Problem: Given a page reference string: **7, 0, 1, 2, 0, 3, 0, 4, 2, 3** and 3 physical memory page frames (initially empty). Compute the total page faults under FIFO and LRU (**Least Recently Used**).

1. First-In First-Out (FIFO) Replacement:

Ref	Frame 1	Frame 2	Frame 3	Status / Fault
7	7	-	-	Page Fault (1)
0	7	0	-	Page Fault (2)
1	7	0	1	Page Fault (3)
2	2	0	1	Page Fault (4) — Replaces 7 (oldest)
0	2	0	1	Hit
3	2	3	1	Page Fault (5) — Replaces 0
0	2	3	0	Page Fault (6) — Replaces 1
4	4	3	0	Page Fault (7) — Replaces 2
2	4	2	0	Page Fault (8) — Replaces 3
3	4	2	3	Page Fault (9) — Replaces 0

Total FIFO Page Faults: 9 Faults (Suffers from *Belady's Anomaly* where adding frames can increase faults).

2. Least Recently Used (LRU) Replacement:

Ref	Frame 1	Frame 2	Frame 3	Status / Fault
7	7	-	-	Page Fault (1)
0	7	0	-	Page Fault (2)
1	7	0	1	Page Fault (3)
2	2	0	1	Page Fault (4) — Replaces 7 (least recently used)
0	2	0	1	Hit (0 becomes most recent)
3	2	0	3	Page Fault (5) — Replaces 1
0	2	0	3	Hit
4	4	0	3	Page Fault (6) — Replaces 2
2	4	0	2	Page Fault (7) — Replaces 3
3	4	3	2	Page Fault (8) — Replaces 0

Total LRU Page Faults: 8 Faults (LRU belongs to the stack algorithm class and never exhibits Belady's Anomaly).

13.2 Operating System Classifications

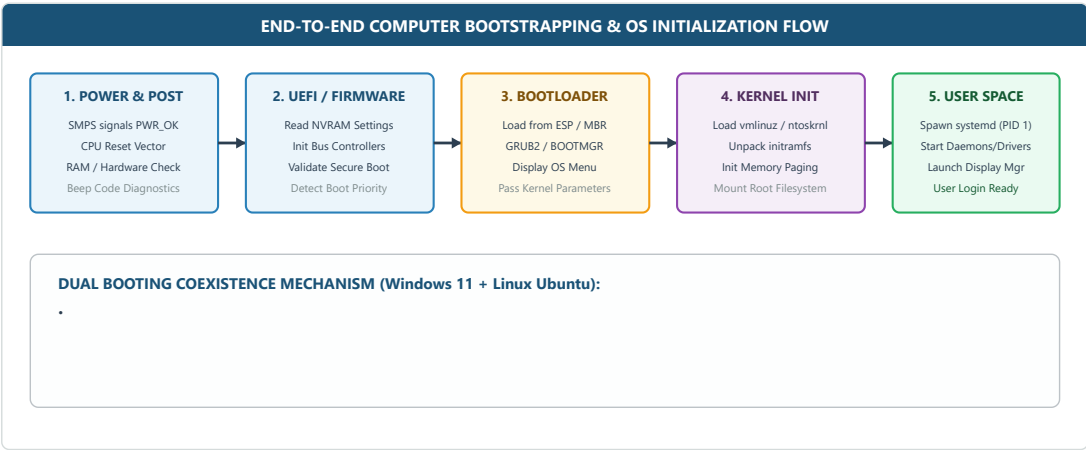
- **Batch Operating Systems:** Early non-interactive systems where jobs with similar requirements were batched together onto magnetic media and executed sequentially by an operator (e.g., IBM OS/360).
- **Multiprogramming Systems:** Increases CPU utilization by keeping multiple jobs in memory simultaneously. When the active job initiates an I/O wait, the OS switches the CPU to another ready job.
- **Time-Sharing (Multitasking) Systems:** Allocates CPU execution time into millisecond time slices (quantums) across multiple active processes via Round Robin scheduling, providing interactive responsiveness for multiple concurrent users.
- **Multiprocessing Systems:** Utilizes two or more physical CPUs or multi-core processors sharing common memory (Symmetric Multiprocessing — SMP) to achieve true parallel execution.

- **Distributed Systems:** Manages a network of physically separated computational nodes communicating over networks, appearing to end-users as a single unified computing facility.
- **Real-Time Operating Systems (RTOS):** Engineered for applications where correctness depends not only on logical output but on meeting strict deterministic timing deadlines:
 - **Hard RTOS:** Missing a single deadline causes total system failure (e.g., pacemaker control, automotive airbag deployment, flight avionics).
 - **Soft RTOS:** Missing a deadline degrades quality of service without system failure (e.g., video streaming, audio playback).

13.3 Kernel Architectures: Monolithic vs Microkernel vs Hybrid

- **Monolithic Kernel (e.g., Linux, FreeBSD):** Entire operating system core — Process Scheduler, Virtual Memory, File Systems, Network Stacks, and all Device Drivers — executes inside a single shared, highly privileged address space in **Ring 0**.
 - *Advantages:* Maximum performance; inter-module communication occurs via ultra-fast direct C function calls.
 - *Disadvantages:* Large kernel codebase; a bug or memory corruption in any third-party driver can crash the entire system.
- **Microkernel (e.g., Minix, QNX, seL4):** Only the absolute bare essentials (Thread Scheduling, Low-level IPC, Address Space translation) run in Ring 0. All other services (File System, Networking, Device Drivers) run as isolated user-mode server processes in **Ring 3**.
 - *Advantages:* High reliability, modularity, and fault tolerance; crashing drivers do not crash the system.
 - *Disadvantages:* Performance overhead caused by frequent user/kernel context switches and message-passing IPC.
- **Hybrid Kernel (e.g., Windows NT, Apple macOS XNU):** Combines monolithic speed with microkernel modularity, running subsystems in kernel space while maintaining modular driver interfaces.

13.4 Power-On Self-Test (POST) & Bootstrapping Sequence



EFI System Partition (ESP): Formatted as FAT32 (100–512 MB), houses \EFI\Microsoft\Boot\bootmgfw.efi and \EFI\ubuntu\grubx64.efi. • **GRUB2 Boot Manager:** Scans storage partitions via `os-prober`, detects Windows BCD, and presents user with interactive OS selection. • **Critical Setting:** Disable Windows "Fast Startup" in Control Panel to prevent NTFS partition dirty-bit locking and data corruption.

Figure 3.2: Complete System Bootstrapping Sequence and Dual-Boot Architecture.

Detailed Execution Steps of System Bootstrapping:

1. **Power Sequencing & Reset Vector:** The SMPS asserts `PWR_OK`. The CPU resets its registers and jumps to the hardcoded Reset Vector (x86: physical address `0xFFFFFFF0`), executing the initial jump to UEFI Flash ROM code.
2. **Power-On Self-Test (POST):** Low-level firmware routines test CPU arithmetic logic, ROM checksums, RTC/NVRAM clocks, DRAM data lines, and enumerate PCIe devices. Hardware errors are signaled via piezoelectric beep codes or motherboard 2-digit 7-segment hex displays (Port 80h).
3. **Firmware Initialization & Boot Selection:** UEFI initializes bus controllers, validates Secure Boot digital signatures, reads boot priority from NVRAM, and loads the target EFI bootloader from the EFI System Partition (ESP).
4. **Bootloader Handover:** Windows `bootmgfw.efi` or Linux `grubx64.efi` executes, loads kernel binary images into RAM (`ntoskrnl.exe` or `vmlinuz`), and passes hardware configuration parameters.
5. **Kernel Initialization:** The kernel initializes memory management paging, unpacks the initial RAM disk (`initramfs`), mounts the root filesystem, initializes built-in drivers, and spawns the first user-space initialization process (`systemd` on Linux with PID 1 / `smss.exe` on Windows).

Unit 14: OS Installation, Partitioning, Boot Managers & File Systems

14.1 Bootable Media Creation & Installation Workflow

Operating system installation begins with creating bootable installation media (typically a USB flash drive).

- **ISO Image Architecture:** An uncompressed optical disc sector image archive (ISO 9660 / UDF format) containing the entire OS installer, compressed kernel squashfs images, package repositories, and bootable El Torito boot records.
- **Flashing Utilities (Rufus, Balena Etcher, Ventoy):** Software tools that unpack the ISO and write raw sector boot blocks onto the USB drive. When using Rufus, users configure target partition scheme (MBR or GPT) and target system firmware (BIOS or UEFI).

14.2 Disk Partitioning: MBR vs GPT

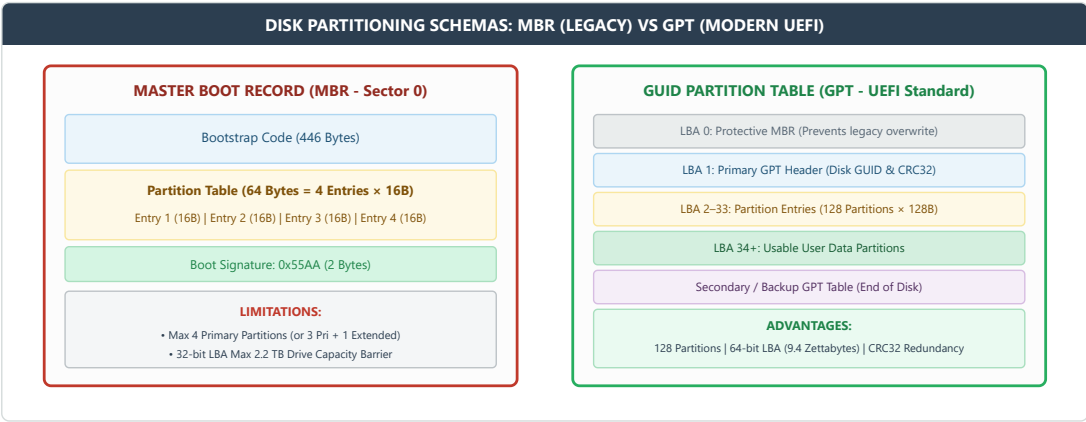


Figure 3.3: Architectural Layout of MBR Partition Table vs Modern GPT Schema.

1. Master Boot Record (MBR - Legacy Schema)

Resides at Sector 0 (exactly 512 bytes):

- **Bootstrap Code (446 Bytes):** Contains the Stage-1 machine code executed by BIOS to scan the partition table, find the partition flagged as active/bootable (`0x80`), and load its Volume Boot Record (VBR).
- **Partition Table (64 Bytes):** Exactly four 16-byte partition records. Each record stores:
 - Byte 0: Boot indicator (`0x80` = Active bootable, `0x00` = Inactive).
 - Bytes 1–3: Starting Cylinder-Head-Sector (CHS) coordinates.
 - Byte 4: Partition Type ID (e.g., `0x07` for NTFS, `0x83` for Linux native).
 - Bytes 5–7: Ending CHS coordinates.
 - Bytes 8–11: Starting Logical Block Address (LBA) sector number (32-bit integer).
 - Bytes 12–15: Total sector count in the partition (32-bit integer).
- **Boot Signature (2 Bytes):** Hex word `0x55AA` verifying sector validity.
- **The 2.2 TB Capacity Barrier:** MBR uses 32-bit Logical Block Addressing (LBA) sector numbers. With standard 512-byte sectors:

$$\text{Max MBR Size} = 2^{32} \times 512 \text{ bytes} = 2,199,023,255,552 \text{ bytes} \approx \textbf{2.19 TB}$$

Any storage capacity exceeding **2.2 TB** on an MBR disk is completely unaddressable.

2. GUID Partition Table (GPT - Modern UEFI Schema)

Developed as part of the Unified EFI standard, GPT replaces legacy MBR:

- **64-Bit LBA Addressing:** Supports drive capacities up to $2^{64} \times 512 \text{ bytes} \approx \textbf{9.4 Zettabytes}$ ($9.4 \times 10^{21} \text{ bytes}$).
- **Partition Count:** Supports a minimum of **128 distinct primary partitions** without needing extended/logical partition hacks.
- **Data Redundancy & Integrity:** GPT stores a Primary Header and Partition Table at the beginning of the disk (LBA 1–33) and a **Secondary Backup GPT Header** at the trailing end of the disk. Every partition entry and header is protected by **CRC32 Checksums**; if corruption occurs, UEFI automatically restores the partition table from the backup copy.
- **Protective MBR:** GPT places a dummy MBR at LBA 0 with a single partition type `0xEE` spanning the entire drive, preventing legacy MBR-only utilities from misidentifying the disk as unpartitioned and destroying user data.

14.3 Boot Managers: Windows BOOTMGR vs GNU GRUB2

- **Windows Boot Manager (`BOOTMGR`):** The proprietary bootloader for Windows. Reads configuration data from the **Boot Configuration Data (BCD)** registry hive, verifies driver digital signatures, loads `winload.efi` (or `winload.exe`), initializes the Windows Kernel (`ntoskrnl.exe`), and transfers execution.
- **GNU GRand Unified Bootloader (GRUB2):** The dominant open-source multi-boot loader in Linux environments:
 - **Stage 1 (`boot.img`):** Tiny 512-byte code residing in MBR sector 0 or loaded by UEFI.
 - **Stage 1.5 / Core (`core.img`):** Contains file system drivers (ext4, NTFS, FAT32) allowing GRUB to access Stage 2 files.
 - **Stage 2 (`/boot/grub/grub.cfg`):** Displays the interactive multi-boot menu, allows runtime editing of kernel boot parameters, unpacks the `initramfs` RAM disk, loads the compressed Linux kernel (`vmlinuz`), and passes system control.

14.4 Dual Booting Architecture (Windows & Linux)

Dual booting enables a single physical computer to host multiple independent operating systems selectable via a boot menu at startup.

- **Partition Layout:**
 - **EFI System Partition (ESP):** Formatted strictly as **FAT32 (100–512 MB)**. Contains `\EFI\Microsoft\Boot\bootmgfw.efi` and `\EFI\ubuntu\grubx64.efi` simultaneously.
 - **Windows C: Partition:** Formatted as NTFS (**100+ GB**).
 - **Linux Root Partition (/):** Formatted as ext4 (**50+ GB**).
 - **Linux Swap Partition:** Dedicated virtual memory paging space.
- **Chainloading Mechanism:** During UEFI boot, GRUB is executed as the primary boot manager. GRUB executes its `os-prober` script, discovers the Windows BCD entry on the ESP, and offers an interactive choice. Selecting Windows "chainloads" execution directly to `bootmgfw.efi`.
- **Windows Fast Startup Caveat:** Windows "Fast Startup" puts the OS into a hybrid hibernation state that locks the NTFS file system metadata dirty-bit. If Linux mounts that NTFS partition while Fast Startup is active, severe disk corruption occurs. Fast Startup **must be disabled** in dual-boot setups.

14.5 File Systems: FAT vs NTFS vs ext4

Feature / Metric	FAT32	exFAT	NTFS	ext4
Primary OS	Legacy Windows / Flash	Universal Flash Media	Modern Windows (10/11)	Linux Distributions
Max Single File Size	4 GB – 1 Byte	16 Exabytes	16 Terabytes	16 Terabytes
Max Volume Size	2 TB to 16 TB	128 Petabytes	8 Petabytes	1 Exabyte
Journaling Support	No (Prone to corruption)	No	Yes (USN Journal)	Yes (Superblock + Extents)
File Permissions & ACLs	None	None	Granular Windows ACLs	POSIX (rwx, User/Group/Other)
Compression / Encryption	No	No	Native NTFS Compression & EFS	Native transparent encryption

Internal File System Architectures & Journaling Modes:

- **FAT32 Cluster Chains:** Files are stored as linked lists of clusters in the File Allocation Table (FAT). Each entry contains a 28-bit cluster index pointing to the next cluster, with `0x0FFFFFFF` indicating End-Of-File (EOF). Because file access requires sequentially traversing the FAT chain, random access on fragmented drives suffers significant performance degradation.
- **NTFS (Master File Table - MFT):** Everything on an NTFS volume is a file, including metadata files (e.g., `$MFT`, `$LogFile`, `$Bitmap`). Each file has a 1024-byte record in the MFT containing resident data or non-resident data run-lists. Directories are indexed using balanced B-Trees for rapid $O(\log n)$ file lookups.
- **ext4 (Inodes & Extent Trees):** An ext4 filesystem divides the disk into **Block Groups**. Each file is represented by an **Inode** containing file metadata (size, timestamps, POSIX permissions, owner UID/GID) and an **Extent Tree** (a contiguous run of physical blocks represented by starting block and length), replacing legacy indirect block pointers to drastically reduce disk fragmentation. ext4 supports three journaling modes:
 - *Journal:* Both metadata and file data are written to the journal before committing to disk (highest integrity, lowest write speed).
 - *Ordered:* Only metadata is journaled, but file data blocks are flushed to disk before committing metadata (default Linux mode).
 - *Writeback:* Metadata is journaled without ordering guarantees for data blocks (highest performance, risk of stale data on crash).

Unit 15: Device Drivers & Kernel Subsystems

15.1 Need and Role of Device Drivers

A **Device Driver** is a specialized system software program that serves as a translator and abstraction bridge between the generic I/O subsystem of the Operating System kernel and the specific electrical, register-level hardware mechanics of physical peripheral devices.

WHY DEVICE DRIVERS ARE INDISPENSABLE

Hardware manufacturers build peripherals (Realtek audio chips, Intel Wi-Fi cards, NVIDIA GPUs, HP printers) with wildly differing internal register layouts, command timing sequences, and physical firmware interfaces. The OS kernel cannot include hardcoded machine logic for millions of peripheral models. Instead, the OS defines standardized abstract I/O interfaces (e.g., `read()`, `write()`, `open()`, `close()`, `ioctl()`). The device driver implements these standardized calls by issuing device-specific commands to hardware registers.

15.2 Protection Rings & Driver Execution Modes

Modern x86/x64 processors implement hardware-enforced **Hierarchical Protection Rings** to isolate software components:

- **Ring 0 (Supervisor / Kernel Mode):** Highest privilege level. Code in Ring 0 has unrestricted direct access to all CPU instructions (including privileged instructions like `CLI`, `STI`, `IN`, `OUT`, `LIDT`), all physical memory addresses, and all I/O hardware ports.
 - **Kernel-Mode Device Drivers:** Most high-performance drivers (GPU display drivers, storage controllers, network cards) execute in Ring 0. *Risk:* A single null-pointer dereference or unhandled exception in a Kernel-Mode driver immediately crashes the entire operating system, resulting in a **Blue Screen of Death (BSOD)** in Windows or a **Kernel Panic** in Linux.

- **Ring 3 (User Mode):** Lowest privilege level. Standard user applications execute here with restricted virtual memory access. Direct hardware I/O instructions are strictly prohibited; any unauthorized attempt raises a hardware General Protection Fault (GPF).
 - **User-Mode Driver Framework (UMDF):** Modern OS designs execute non-time-critical drivers (software printers, USB sensors, smart card readers) in Ring 3 isolated worker processes. If a User-Mode driver crashes, the OS simply restarts that specific process without affecting system stability.

15.3 Driver-Hardware Interaction Mechanisms

Device drivers coordinate hardware communication through four foundational mechanisms:

1. **Port-Mapped I/O (PMIO):** The CPU maintains a dedicated 16-bit physical I/O port address space separate from system RAM, accessed via specialized assembly instructions: `IN AL, DX` (read byte from port in `DX`) and `OUT DX, AL` (write byte to port). Legacy x86 architecture used by legacy keyboards (Port 60h/64h), floppy disk controllers (Port 3F0h), and serial COM ports.
2. **Memory-Mapped I/O (MMIO):** Modern high-speed standard where peripheral device control registers and internal buffers are mapped directly into the physical memory address space of the CPU. The driver reads and writes to device hardware using standard memory instructions (e.g., `MOV`), benefiting from CPU memory caching, speculative execution pipelines, and high-speed bus interconnects.
3. **Interrupts, APIC, MSI-X & ISR Architecture:**
 - *Legacy PIC vs Modern APIC:* Legacy systems routed 15 IRQ lines through two cascaded Intel 8259A Programmable Interrupt Controllers. Modern multi-core processors utilize the **Advanced Programmable Interrupt Controller (APIC)** architecture, consisting of a *Local APIC (LAPIC)* per CPU core and an *I/O APIC* on the motherboard chipset, supporting up to 240 interrupt vectors per core with inter-processor interrupts (IPI).
 - *Message Signaled Interrupts (MSI / MSI-X):* PCIe devices eliminate dedicated physical interrupt sideband wires by writing special 32-bit/64-bit address data packets directly to the host memory controller, triggering CPU interrupts. **MSI-X** supports up to 2,048 individual interrupt vectors per device, allowing multi-queue NVMe drives and 100GbE NICs to bind separate hardware interrupt queues directly to individual CPU cores for zero-contention parallel processing.
 - *Top-Half vs Bottom-Half Processing:* To prevent stalling the CPU, modern drivers divide interrupt handling: the *Top-Half (ISR)* runs immediately with hardware interrupts disabled to acknowledge the peripheral, clear hardware status bits, and copy packet/block buffers into RAM; the *Bottom-Half (Deferred Procedure Call DPC in Windows / Tasklets & Workqueues in Linux)* processes the bulk data asynchronously at lower interrupt request levels (IRQL) with hardware interrupts re-enabled.
4. **Direct Memory Access (DMA) & Scatter-Gather Descriptors:**
 - Without DMA, the CPU would be forced to read every incoming data byte from hardware into a CPU register and then write it to RAM (Programmed I/O - PIO), wasting 100% of CPU cycles and throttling system throughput.
 - A **DMA Controller (DMAC)** allows high-speed peripherals (NVMe SSDs, Gigabit NICs) to transfer massive data blocks directly into and out of system RAM over PCIe/system buses without CPU intervention. The driver initializes physical source/destination addresses and byte length, and the DMAC transfers data autonomously at memory bus speeds.
 - **Scatter-Gather DMA:** Real-world operating systems allocate virtual memory in non-contiguous physical 4 KB pages. Standard DMA would require expensive memory buffer copies into contiguous bounce buffers. With *Scatter-Gather DMA*, the driver constructs a linked list of physical memory descriptors (called a **Scatter-Gather List - SGL**), where each descriptor contains a 64-bit physical memory pointer and byte length. The hardware DMAC traverses the descriptor chain automatically, reading from or writing to discontinuous physical RAM pages in a single uninterrupted hardware transaction.

15.4 Linux Kernel Module (LKM) & Windows Driver Framework (WDF)

- **Linux Loadable Kernel Modules (LKM):** LKMs are dynamically linked into the running kernel without rebooting using user-space utilities (`insmod`, `modprobe`, `rmmod`). LKMs provide standard entry/exit points (`module_init()` and `module_exit()`) and interface with user-space via Character (`/dev/chr`), Block (`/dev/sda`), or Network (`eth0`) abstractions.

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Computer Science Faculty");
MODULE_DESCRIPTION("Sample Linux Character Device Driver");

static int device_open(struct inode *inode, struct file *file) {
    pr_info("Device opened successfully\n");
    return 0;
}

static struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = device_open,
};

static int __init sample_driver_init(void) {
    pr_info("Registering character device driver\n");
    return register_chrdev(240, "sample_dev", &fops);
}

static void __exit sample_driver_exit(void) {
    pr_info("Unregistering character device driver\n");
    unregister_chrdev(240, "sample_dev");
}
```

```
module_init(sample_driver_init);  
module_exit(sample_driver_exit);
```

- **Windows Driver Framework (WDF):** Modern unified driver architecture in Windows:
 - *Kernel-Mode Driver Framework (KMDF):* Object-oriented C-based framework handling I/O Queuing, Plug and Play (PnP), and Power Management state machines for Ring 0 drivers. Handles I/O Request Packets (IRPs) through structured dispatch tables.
 - *User-Mode Driver Framework (UMDF):* COM-based C++ framework running drivers inside isolated user-mode Host Processes (`WUDFHost.exe`). If a user-mode driver encounters an unhandled exception, the OS restarts the worker process without destabilizing the Windows kernel.

Fundamentals of Computers and Computational Thinking

Course Code: CS101 Credits: 4 (Lecture: 3, Practical: 2) Total Hours: 75 Hrs

MODULE IV: Computational Thinking, Logic and Algorithms

Comprehensive Academic Notes — Units 16 through 22 (Duration: 13 Hours, Weightage: 20 Marks)

Unit 16: Computer Science as a Discipline — Scope & Grand Challenges

CONCEPTUAL DEFINITION: COMPUTER SCIENCE

Computer Science is the systematic study of algorithmic processes, computational machines, information structures, and the theoretical foundations of automation, encompassing software design, hardware architecture, complexity analysis, and mathematical modeling of computation.

Computer Science is not merely the study of computer hardware or programming syntax; as Edsger W. Dijkstra famously noted, "*Computer science is no more about computers than astronomy is about telescopes.*" It represents the formal science of problem-solving, information transformation, and the limits of automated computability. The discipline investigates three foundational questions: (1) *What can be computed?* (Computability Theory), (2) *How efficiently can it be computed?* (Computational Complexity), and (3) *How can practical physical and logical architectures be engineered to perform these computations reliably?* (Computer Systems and Software Engineering).

16.1 Sub-disciplines of Computer Science

- **Theoretical Computer Science:** Automata Theory, Formal Languages, Turing Machines, Computability, and Computational Complexity Classes (***P* vs *NP***, ***NP*-Complete**). Investigates fundamental questions regarding what can be solved within finite polynomial time versus what is undecidable (e.g., Turing's Halting Problem).
- **Computer Systems and Architecture:** Microprocessor Microarchitectures, Multi-core Symmetric Multiprocessing (SMP), Instruction-Level Parallelism (ILP), Distributed Systems, Operating Systems, Computer Networks, and Real-Time Systems.
- **Software Engineering:** Methodologies, design patterns, and formal verification frameworks for designing, architecting, implementing, testing, and maintaining large-scale, robust, fault-tolerant software systems.
- **Artificial Intelligence and Machine Learning:** Artificial Neural Networks, Deep Learning, Natural Language Processing, Computer Vision, Robotics, Knowledge Representation, and Autonomous Agent systems.
- **Data Science and Cryptography:** Big Data Analytics, Database Management Engines, Relational and NoSQL storage models, Cryptographic Protocols, Information Security, and Post-Quantum Cryptography.

16.2 Mathematical Foundations: Automata & The Turing Machine

In 1936, Alan Turing introduced the **Turing Machine (TM)**, a mathematical model of computation that defines the theoretical limits of automated computation. A Turing machine consists of an infinite memory tape divided into discrete cells, a read/write tape head that can scan, modify, and move left or right across tape cells, and a finite state control unit.

Formally, a deterministic Turing machine is defined as a 7-tuple:

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}} \rangle$$

where:

- Q is a finite non-empty set of internal machine states.
- Σ is the finite input alphabet not containing the blank symbol $\sqcup$.
- Γ is the finite tape alphabet ($\Sigma \subset \Gamma$ and $\sqcup \in \Gamma$).
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function mapping the current state and scanned tape symbol to a next state, a replacement symbol to write on the tape, and a directional tape head movement (Left or Right).
- $q_0 \in Q$ is the initial start state.
- $q_{\text{accept}} \in Q$ is the accepting halting state.
- $q_{\text{reject}} \in Q$ is the rejecting halting state ($q_{\text{accept}} \neq q_{\text{reject}}$).

The **Church-Turing Thesis** posits that any algorithmic calculation that can be naturally performed by a human mathematician following a deterministic effective procedure can be simulated by a Universal Turing Machine. This establishes the theoretical equivalence between Turing Machines, Lambda Calculus, and modern digital electronic computers.

16.3 Grand Challenges & Turing Award Milestones

The discipline is defined by historic milestones recognized by the ACM A.M. Turing Award (the "Nobel Prize of Computing"):

- **Alan Turing (1936):** Universal Turing Machine and Computability foundations. Proved the undecidability of the Halting Problem.
- **John von Neumann (1945):** Stored-program computer architecture unifying code and data in common addressable memory.

- **Claude Shannon (1948):** Mathematical Theory of Communication and Information Entropy ($H = -\sum p_i \log_2 p_i$).
- **Donald Knuth (1974):** Formalization of Algorithm Analysis, asymptotic notations, and *The Art of Computer Programming*.
- **Stephen Cook (1971) & Richard Karp (1972):** Foundations of **NP**-Completeness, Cook-Levin Theorem, and the 21 Karp reductions.

THEORETICAL PROOF: TURING'S HALTING PROBLEM IS UNDECIDABLE

Theorem: There exists no general algorithm that can determine whether an arbitrary program P will eventually halt (terminate) or run forever on an arbitrary input I .

Proof by Contradiction (Diagonalization):

1. Assume such a halting decider exists: a program $\text{Halt}(P, I)$ that returns `True` if $P(I)$ halts and `False` if $P(I)$ loops forever.
2. Construct an adversary program $\text{TuringOpposite}(P)$:

```
def TuringOpposite(P):
    if Halt(P, P) == True:
        while True: pass # Loop infinitely
    else:
        return True      # Halt immediately
```

3. What happens when we execute $\text{TuringOpposite}(\text{TuringOpposite})$?
 - If $\text{Halt}(\text{TuringOpposite}, \text{TuringOpposite})$ returns `True` (halts), TuringOpposite enters an infinite loop (does not halt).
 - If $\text{Halt}(\text{TuringOpposite}, \text{TuringOpposite})$ returns `False` (loops forever), TuringOpposite returns immediately (halts).
4. In both cases, we reach a direct logical contradiction ($P \iff \neg P$).

Conclusion: The initial assumption that a general halting decider exists is false. The Halting Problem is undecidable.

16.4 Complexity Classes: P , NP , NP -Complete, and NP -Hard

- **Class P (Polynomial Time):** The set of all decision problems that can be solved by a deterministic Turing machine in polynomial time $O(n^k)$ for some constant k (e.g., Shortest Path via Dijkstra, Sorting, Linear Search).
- **Class NP (Nondeterministic Polynomial Time):** The set of decision problems for which a candidate solution can be *verified* in polynomial time by a deterministic machine, or solved in polynomial time by a nondeterministic machine (e.g., Traveling Salesperson Problem, Boolean Satisfiability SAT, Subset Sum).
- **Class NP -Complete (NPC):** A problem X is NP -Complete if $X \in NP$ and every problem in NP can be polynomial-time reduced to X ($Y \leq_p X \forall Y \in NP$). If any single NP -Complete problem is solved in polynomial time, then $P = NP$.
- **Class NP -Hard:** Problems at least as hard as the hardest problems in NP , but not necessarily in NP (e.g., the Halting Problem, optimization version of TSP).
- **The P versus NP Millennium Problem:** The fundamental question asks: $P \stackrel{?}{=} NP$. If $P = NP$, then every problem whose solution can be quickly verified could also be quickly solved, fundamentally revolutionizing cryptography, artificial intelligence, and mathematics.
- **Post-Quantum Cryptography & Quantum Supremacy:** Shor's Algorithm solves prime factorization and discrete logarithms in polynomial time $O((\log N)^3)$ on a quantum computer, breaking RSA and ECC cryptography. The grand challenge is transitioning global cybersecurity to Lattice-based Post-Quantum Cryptography (e.g., CRYSTALS-Kyber and Dilithium).

Unit 17: Problem Solving Fundamentals & Software Development Lifecycle

CONCEPTUAL DEFINITION: PROBLEM SOLVING IN COMPUTING

Problem Solving in computer science is the systematic, structured cognitive process of analyzing an ill-defined real-world situation, identifying precise input constraints and expected output criteria, and formulating an executable sequence of deterministic operations that transforms inputs into valid solutions.

17.1 Polya's Four-Step Problem-Solving Framework

Renowned mathematician George Polya formulated a universal 4-step framework for mathematical and computational problem solving:

1. **Understand the Problem:** Identify the unknowns, data inputs, operational conditions, and edge-case boundary constraints. Restate the problem in formal terminology, identifying preconditions (P_{pre}) and postconditions (P_{post}).
 - *Example:* Sorting an array requires knowing the data type, whether duplicate elements exist, memory constraints ($O(1)$ auxiliary space vs $O(N)$), and whether stability is required.
2. **Devise a Plan:** Find connections between data and unknowns. Look for familiar patterns, decompose into smaller sub-problems, consider related algorithms (Divide-and-Conquer, Greedy, Dynamic Programming), or formulate a mathematical recurrence relation.
3. **Carry Out the Plan:** Implement the algorithmic steps into clean, structured pseudocode or high-level code, checking each step for logical correctness, loop invariants, and boundary safety.

4. **Look Back (Review & Refine):** Examine the resulting solution. Can you derive the result differently? Is the time and space complexity optimal? Does it handle extreme edge cases ($N = 0$, negative inputs, arithmetic overflow)?

17.2 The Input-Process-Output (IPO) Architecture

Every computational system, from a simple calculator to a multi-node cloud platform, fundamentally operates on the **Input-Process-Output (IPO)** model:

- **Input Phase (I):** Acquisition and validation of raw data from external peripherals (keyboard, sensors, network sockets, databases). Requires strict data validation, type checking, range assertion, and sanitization to prevent injection vulnerabilities.
- **Processing Phase (P):** Execution of deterministic algorithmic transformations, mathematical calculations, logical comparisons, data structure manipulations, and state transitions within CPU registers and RAM.
- **Output Phase (O):** Presentation or transmission of structured results to target destinations (monitors, disk storage, actuators, external APIs).

17.3 The 7-Stage Software Development Lifecycle (SDLC)

1. **Problem Definition & Requirements Analysis:** Identifying stakeholder needs, specifying functional and non-functional requirements (SRS document), and establishing formal problem boundaries.
2. **System & Algorithmic Design:** Designing data structures, algorithm steps, modular architecture, and creating flowcharts, pseudocode specifications, and ER diagrams.
3. **Implementation / Coding:** Translating algorithmic blueprints into high-level programming language code (C, Python, Java) adhering to best practices and coding standards.
4. **Verification, Formal Proofs & Testing:**
 - **Hoare Logic & Program Verification Triples:** Formal programs are verified using Hoare triples $\{P\} C \{Q\}$, where P is the precondition asserting state before command C executes, and Q is the postcondition guaranteed upon termination.
 - **Loop Invariant Verification:** Proving algorithm correctness requires a loop invariant assertion I that satisfies three properties:
 1. *Initialization:* I is true prior to the first iteration of the loop.
 2. *Maintenance:* If I is true before an iteration, it remains true before the next iteration.
 3. *Termination:* When the loop terminates, the invariant I combined with the loop termination condition proves the algorithm's postcondition.
 - **Black-Box Testing Techniques:** Testing functional behavior without internal code knowledge:
 - **Equivalence Class Partitioning (ECP):** Divides the input domain into valid and invalid equivalence classes, selecting representative values from each partition.
 - **Boundary Value Analysis (BVA):** Selects test inputs at the extreme boundary edges: **Min, Min + 1, Nominal, Max - 1, Max**
 - **White-Box Testing & Cyclomatic Complexity:** Testing internal code structure, statement coverage, branch coverage, and path coverage. McCabe's Cyclomatic Complexity:

$$V(G) = E - N + 2P$$

where E is the number of control flow edges, N is the number of nodes, and P is the number of connected components. $V(G)$ defines the upper bound on the number of linearly independent execution paths requiring dedicated test cases.

5. **Deployment:** Releasing the verified binary executable or system into staging and production operational environments.
6. **Maintenance & Evolution:** Patching newly discovered defects (Corrective), optimizing performance (Perfective), adapting to new OS environments (Adaptive), and adding new features (Preventive).

Comparative Matrix: Software Engineering Methodologies

Methodology	Workflow Structure	Flexibility to Change	Ideal Project Type
Waterfall Model	Strict linear-sequential phases	Very Low (Rigid scope)	Mission-critical systems with unchanging specs (Avionics, Medical)
V-Model (Verification/Validation)	Paired development and testing stages	Low	Safety-critical embedded systems requiring rigorous test plans
Agile / Scrum	Iterative, incremental 2-4 week Sprints	Very High (Continuous feedback)	Commercial web apps, dynamic enterprise software, SaaS platforms

Unit 18: Four Pillars of Computational Thinking

CONCEPTUAL DEFINITION: COMPUTATIONAL THINKING

Computational Thinking (CT) is a formal thought process involved in formulating problems and their solutions in such a way that the solutions are represented in a form that can be effectively carried out by an information-processing agent (human or computer). Coined and popularized by Professor Jeannette Wing (2006).

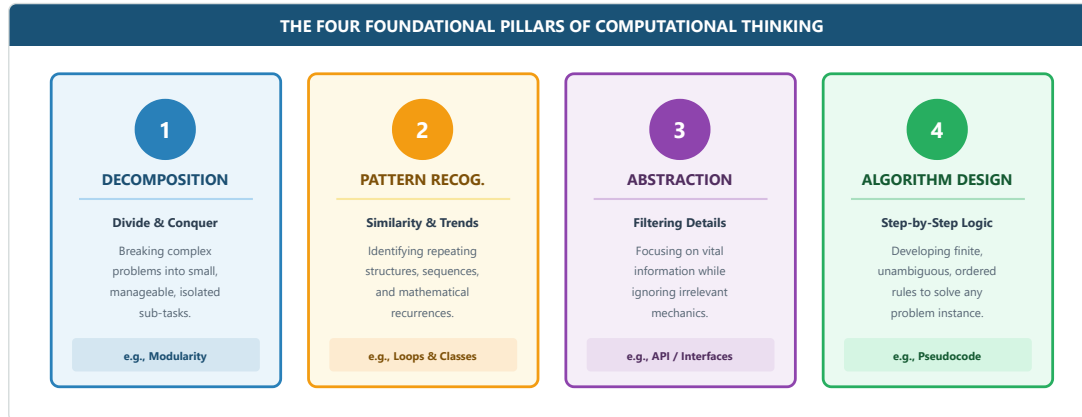


Figure 4.1: The Four Pillars of Computational Thinking.

18.1 Deep Dive into the Four Pillars with Real-World Case Studies

- Decomposition:** The analytical process of breaking down a large, complex, monolithic problem into smaller, self-contained, easily solvable sub-problems.
 - Case Study 1 (Autonomous Vehicle Navigation):* A self-driving car system is too complex for a single program. Decomposition divides the system into: (a) Computer Vision & Sensor Fusion (processing camera, LIDAR, and radar streams), (b) Localization (GPS and IMU coordinate tracking), (c) Motion & Path Planning (Dijkstra / A* trajectory calculations), and (d) Actuation Control (CAN-bus steering, acceleration, and braking).
 - Case Study 2 (Compiler Engineering):* Compilers decompose high-level programming language translation into distinct sequential phases: Lexical Analysis (Scanning source tokens), Syntax Analysis (Context-Free Grammar Parsing into Abstract Syntax Trees), Semantic Analysis (Type checking and scope validation), Intermediate Code Generation (Three-Address Code), Machine-Independent Optimization (Loop unrolling, dead code elimination), and Target Code Generation (Assembly / Machine Instructions).
- Pattern Recognition:** Observing and analyzing similarities, trends, periodic cycles, and recurring patterns within decomposed sub-problems to apply previously verified solutions.
 - Case Study 1 (Dynamic Programming & Bioinformatics):* In computing the global sequence alignment between two DNA strands (e.g., Needleman-Wunsch algorithm), recognizing that optimal alignment of prefix sequences $S_1[1..i]$ and $S_2[1..j]$ depends only on three previously computed adjacent states:

$$M(i, j) = \max \begin{cases} M(i-1, j-1) + \text{Score}(S_1[i], S_2[j]) \\ M(i-1, j) + \text{GapPenalty} \\ M(i, j-1) + \text{GapPenalty} \end{cases}$$

allows $O(mn)$ dynamic programming tabulation instead of $O(2^{m+n})$ exponential recursion.

- Case Study 2 (Natural Language Processing):* Recognizing byte-pair encoding (BPE) subword patterns across vocabulary allows LLMs to tokenize arbitrary sentences into numerical embedding vectors efficiently.
- Abstraction:** The process of identifying the essential, high-level features of an object or process while filtering out and ignoring non-essential, low-level implementation details.
 - Case Study (GPS Map Routing):* A city road network is represented as an abstract mathematical Graph $G = (V, E)$ where road intersections are vertices (V) and streets are weighted edges (E). Irrelevant physical details (street tree counts, sidewalk material, building paint colors) are abstracted away, enabling graph algorithms (Dijkstra, Bellman-Ford) to find shortest routes efficiently.
 - Case Study (Abstract Data Types):* Abstracting a `Queue` interface as `enqueue()`, `dequeue()`, and `isEmpty()` hides whether the underlying implementation uses a circular array or a doubly-linked list.
 - Algorithm Design:** Formulating an ordered, deterministic sequence of unambiguous step-by-step instructions that can be mechanically executed to produce the correct solution for every valid input instance.

Unit 19: Logical Reasoning & Propositional Logic

19.1 Modes of Logical Reasoning

- Deductive Reasoning (Top-Down Logic):** Derives guaranteed, logically certain conclusions from established true general premises.

Premise 1: All humans are mortal. Premise 2: Socrates is human. $\implies$ Conclusion: Socrates is mortal.

In programming, deductive reasoning guarantees that if an algorithm's preconditions and invariants hold, its postconditions must hold.

- **Modus Ponens (Affirming the Antecedent):** If $P \rightarrow Q$ is True and P is True, then Q is True ($[(P \rightarrow Q) \wedge P] \implies Q$).
 - **Modus Tollens (Denying the Consequent):** If $P \rightarrow Q$ is True and Q is False ($\neg Q$), then P is False ($\neg P$) ($[(P \rightarrow Q) \wedge \neg Q] \implies \neg P$).
- **Inductive Reasoning (Bottom-Up Logic):** Formulates broad general hypotheses and probabilistic rules from specific empirical observations.

Observation: The sun rose in the east today, yesterday, and every recorded day. $\implies$ **Hypothesis: The sun always rises in the east.**

Serves as the foundation of Machine Learning and Statistical Artificial Intelligence.

- **Abductive Reasoning:** Starts with an incomplete set of observations and proceeds to the most likely or simplest explanation (diagnostic debugging).

19.2 Propositional Logic & Truth Tables

A **Proposition** is a declarative statement that is either strictly **True (T/1)** or strictly **False (F/0)**, but never both simultaneously.

Fundamental Boolean Operators and Truth Table

P	Q	NOT $\neg P$	AND $P \wedge Q$	OR $P \vee Q$	XOR $P \oplus Q$	Implication $P \rightarrow Q$	Biconditional $P \leftrightarrow Q$
0	0	1	0	0	0	1	1
0	1	1	0	1	1	1	0
1	0	0	0	1	1	0	0
1	1	0	1	1	0	1	1

Fundamental Laws of Boolean Algebra:

- **De Morgan's Laws:**

$$\neg(P \wedge Q) \iff \neg P \vee \neg Q$$

$$\neg(P \vee Q) \iff \neg P \wedge \neg Q$$

Algebraic Proof of $\overline{X \cdot Y} = \bar{X} + \bar{Y}$ using Huntington Postulates:

1. Show $(X \cdot Y) + (\bar{X} + \bar{Y}) = 1$:

$$(X \cdot Y) + (\bar{X} + \bar{Y}) = ((X \cdot Y) + \bar{X}) + \bar{Y} = (X + \bar{X})(Y + \bar{X}) + \bar{Y} = 1 \cdot (Y + \bar{X}) + \bar{Y} = (Y + \bar{Y}) + \bar{X} = 1 + \bar{X} = 1$$

2. Show $(X \cdot Y) \cdot (\bar{X} + \bar{Y}) = 0$:

$$(X \cdot Y) \cdot \bar{X} + (X \cdot Y) \cdot \bar{Y} = (X \cdot \bar{X}) \cdot Y + X \cdot (Y \cdot \bar{Y}) = 0 \cdot Y + X \cdot 0 = 0 + 0 = 0$$

Since $\bar{X} + \bar{Y}$ satisfies both complementation axioms with respect to $(X \cdot Y)$, by uniqueness of complements, $\overline{X \cdot Y} = \bar{X} + \bar{Y}$.

- **Distributive Laws:**

$$P \wedge (Q \vee R) \iff (P \wedge Q) \vee (P \wedge R)$$

$$P \vee (Q \wedge R) \iff (P \vee Q) \wedge (P \vee R)$$

- **Absorption Laws:**

$$P \vee (P \wedge Q) \iff P, \quad P \wedge (P \vee Q) \iff P$$

- **Consensus Theorem:**

$$XY \vee \bar{X}Z \vee YZ \iff XY \vee \bar{X}Z$$

Proof: $XY + \bar{X}Z + YZ(X + \bar{X}) = XY + \bar{X}Z + XYZ + \bar{X}YZ = XY(1 + Z) + \bar{X}Z(1 + Y) = XY(1) + \bar{X}Z(1) = XY + \bar{X}Z$.

19.3 Karnaugh Maps (K-Maps) for Boolean Minimization

A **Karnaugh Map (K-Map)** is a graphical visual method for minimizing Boolean expressions into optimal Sum-of-Products (SOP) or Product-of-Sums (POS) forms without algebraic trial and error. Cells are arranged in a 2D grid where adjacent cells differ by exactly one binary bit (using **Gray Code ordering: 00, 01, 11, 10**). Groups of 2^k adjacent ones (**1, 2, 4, 8, 16**) are circled to eliminate redundant Boolean literals.

WORKED EXAMPLE 19.1: 3-VARIABLE K-MAP MINIMIZATION

Minimize the Boolean function $F(A, B, C) = \sum m(1, 3, 5, 7) + \sum m(4, 5)$.

Minterm Expansion:

$$F = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + ABC + A\bar{B}\bar{C}$$

K-Map Grid (Rows: $A \in \{0, 1\}$, Columns: $BC \in \{00, 01, 11, 10\}$):

- Row 0 ($A = 0$): Cell 01 (m_1) = 1, Cell 11 (m_3) = 1.
- Row 1 ($A = 1$): Cell 00 (m_4) = 1, Cell 01 (m_5) = 1, Cell 11 (m_7) = 1.

Grouping:

- Quad Group:** m_1, m_3, m_5, m_7 forms a 2×2 quad covering Column $C = 1$. Common literal = C .
- Pair Group:** m_4, m_5 forms a horizontal pair in Row 1 ($A = 1, B = 0$). Common literal = $A\bar{B}$.

Minimal Sum-of-Products (SOP) Expression: $F(A, B, C) = C + A\bar{B}$.

WORKED EXAMPLE 19.2: 4-VARIABLE K-MAP WITH DON'T CARE CONDITIONS

Minimize $F(A, B, C, D) = \sum m(0, 2, 8, 10, 14) + d(5, 15)$.

K-Map Grid (Rows: $AB \in \{00, 01, 11, 10\}$, Columns: $CD \in \{00, 01, 11, 10\}$):

- Row 00: $m_0(0000) = 1, m_2(0010) = 1$
- Row 01: $d_5(0101) = X$
- Row 11: $m_{14}(1110) = 1, d_{15}(1111) = X$
- Row 10: $m_8(1000) = 1, m_{10}(1010) = 1$

Optimal Grouping:

- Corner Quad:** Cells (0, 2, 8, 10) at the four corners of the map wrap around vertically and horizontally. Common literals: $B = 0, D = 0 \implies \bar{B}\bar{D}$.
- Pair Group:** Cells 14, 15 in Row 11 ($AB = 11, C = 1$). Common literals: ABC .

Minimal SOP Expression: $F(A, B, C, D) = \bar{B}\bar{D} + ABC$.

Unit 20: Algorithm Design & Complexity Analysis

CONCEPTUAL DEFINITION: ALGORITHM

An **Algorithm** is a finite, well-defined sequence of unambiguous, computer-implementable instructions designed to solve a specific class of problems or perform a computation.

20.1 Formal Characteristics of an Algorithm (Donald Knuth)

- Input:** An algorithm must have zero or more well-defined externally supplied quantities.
- Output:** An algorithm must produce at least one well-defined result related to the inputs.
- Definiteness (Unambiguity):** Every single step must be clear, precise, and have only one possible interpretation.
- Finiteness:** The algorithm must terminate after a finite number of computational steps for all valid inputs.
- Effectiveness:** Every operation must be sufficiently basic that it can in principle be executed exactly in finite time using pencil and paper.

20.2 Asymptotic Notations and Complexity Analysis

Algorithm efficiency is evaluated mathematically through asymptotic analysis of time and space complexity as input size $n \rightarrow \infty$:

- Big-O Notation (O — Asymptotic Upper Bound):** Defines worst-case performance:

$$f(n) = O(g(n)) \iff \exists c > 0, n_0 \geq 1 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \forall n \geq n_0$$

- Big-Omega Notation (Ω — Asymptotic Lower Bound):** Defines best-case performance:

$$f(n) = \Omega(g(n)) \iff \exists c > 0, n_0 \geq 1 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \forall n \geq n_0$$

- Big-Theta Notation (Θ — Asymptotically Tight Bound):** Defines exact growth rate:

$$f(n) = \Theta(g(n)) \iff \exists c_1, c_2 > 0, n_0 \geq 1 \text{ such that } c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0$$

- Little-o Notation (o — Strict Upper Bound):**

$$f(n) = o(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

- **Little-omega Notation (ω — Strict Lower Bound):**

$$f(n) = \omega(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

THE MASTER THEOREM FOR DIVIDE-AND-CONQUER RECURRENCES

For divide-and-conquer recurrences of the form $T(n) = aT(\frac{n}{b}) + f(n)$, where $a \geq 1, b > 1$:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$. (Tree leaves dominate).
2. If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$. (e.g., Merge Sort: $T(n) = 2T(n/2) + \Theta(n) \implies a = 2, b = 2, \log_2 2 = 1 \implies \Theta(n \log n)$).
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$, and regularity condition $af(n/b) \leq cf(n)$ holds for $c < 1$, then $T(n) = \Theta(f(n))$. (Root work dominates).

MATHEMATICAL DERIVATION: MERGE SORT RECURRENCE VIA RECURSION TREE

Recurrence Relation: $T(n) = 2T(n/2) + c \cdot n$, with base case $T(1) = O(1)$.

Recursion Tree Structure:

- **Level 0 (Root):** 1 node of size $n \implies \text{Cost} = c \cdot n$.
- **Level 1:** 2 nodes of size $n/2 \implies \text{Cost} = 2 \times c(n/2) = c \cdot n$.
- **Level 2:** 4 nodes of size $n/4 \implies \text{Cost} = 4 \times c(n/4) = c \cdot n$.
- **Level k :** 2^k nodes of size $n/2^k \implies \text{Cost} = 2^k \times c(n/2^k) = c \cdot n$.

The recursion tree terminates when the subproblem size reaches $n/2^k = 1 \implies k = \log_2 n$.

Total Depth: $\log_2 n + 1$ levels.

Total Cost:

$$T(n) = \sum_{k=0}^{\log_2 n} c \cdot n = c \cdot n(\log_2 n + 1) = cn \log_2 n + cn = \Theta(n \log_2 n)$$

20.3 Classical Algorithmic Paradigms

- **Binary Search ($O(\log n)$):** In a sorted array, repeatedly halves the search interval. If target is less than midpoint, search left half; otherwise search right half. Recurrence: $T(n) = T(n/2) + O(1) \implies T(n) = O(\log n)$. Maximum comparisons: $\lfloor \log_2 n \rfloor + 1$.
- **Bubble Sort ($O(n^2)$):** Iteratively steps through the list, compares adjacent elements and swaps them if they are in the wrong order. Pass i bubbles the i -th largest element to its correct position. Total comparisons = $\sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2}$. Space complexity: $O(1)$ in-place.
- **Selection Sort ($O(n^2)$):** Finds the minimum element from the unsorted subarray and swaps it with the leftmost unsorted element. Number of comparisons is $\frac{n(n-1)}{2} = O(n^2)$ in all cases; swaps are strictly $O(n)$. Space complexity: $O(1)$ in-place.
- **Insertion Sort ($O(n^2)$ worst, $O(n)$ best):** Iteratively builds a sorted subarray by shifting larger elements one position right to insert the current key into its correct relative position. Ideal for nearly-sorted data. Space complexity: $O(1)$ in-place.
- **Merge Sort ($O(n \log n)$):** Divide-and-conquer algorithm dividing array into two halves, recursively sorting both, and merging them in linear $O(n)$ time using $O(n)$ auxiliary memory space.
- **Euclidean Algorithm for GCD ($O(\log(\min(a, b)))$):** Computes the greatest common divisor of two integers a and b ($a > b$) using the principle $\gcd(a, b) = \gcd(b, a \bmod b)$ until remainder is zero.

Unit 21: Flowcharts & Visual Algorithm Modeling

CONCEPTUAL DEFINITION: FLOWCHART

A **Flowchart** is a formalized diagrammatic representation of an algorithm displaying the sequential step-by-step workflow, data transformations, decision branching, and loop iterations using standardized geometric symbols connected by directional flowlines.

21.1 Standard Flowchart Symbols (ISO 5807 / RAPTOR Standard)

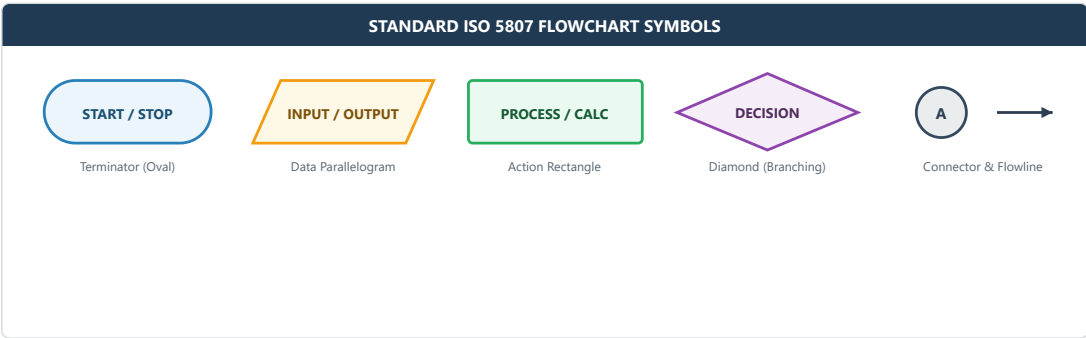


Figure 4.2: Standard Flowchart Graphical Elements for Algorithm Representation.

21.2 Structured Programming Control Constructs

The **Böhm-Jacopini Structured Programming Theorem (1966)** established that any computable algorithm or Turing-computable function can be expressed using only three basic control flow structures without resorting to arbitrary `GOTO` jumps:

- **Sequence Structure:** Linear progression of instructions executed strictly in the order they appear without branching.
- **Selection Structure (Conditional Branching):** Chooses between alternative execution paths based on a Boolean condition:
 - `IF-THEN` (Single alternative)
 - `IF-THEN-ELSE` (Dual alternative)
 - `SWITCH-CASE` (Multi-way branch evaluated via jump table)
- **Iteration Structure (Repetition / Loops):** Repeatedly executes a block of statements while a condition remains true:
 - *Pre-Test Loop* (`WHILE` loop): Condition tested before entering loop body; executes zero or more times.
 - *Post-Test Loop* (`DO-WHILE` loop): Condition tested after executing loop body; executes at least once.
 - *Count-Controlled Loop* (`FOR` loop): Uses initialization, test condition, and step increment variables.

21.3 The RAPTOR Visual Programming Environment

RAPTOR (Rapid Algorithmic Prototyping Tool for Ordered Reasoning) is a visual flowchart-based programming environment developed specifically to help novice computer science students design and execute algorithms without syntax errors. RAPTOR executes flowcharts step-by-step, highlighting active nodes and displaying dynamic variable watches in real time.

Comparative Analysis: Algorithm vs Flowchart vs Pseudocode

Dimension	Natural Language Algorithm	Graphical Flowchart	Structured Pseudocode
Format	Descriptive plain English text sentences	Standardized 2D geometric symbols (ISO 5807)	Language-independent structured text syntax
Visual Scannability	Low; requires reading full text paragraphs	Very High; branches and loops instantly visible	High; indentation reveals block nesting levels
Ease of Modification	High; simple text re-writing	Low; requires redrawing layout lines and shapes	Very High; direct keyboard edits
Translability to Code	Moderate; potential semantic ambiguities	High; maps directly to structured programming	Very High; almost 1-to-1 mapping with C/Python code

Unit 22: Practical Lab Implementations — Flowcharts & Algorithms

Comprehensive step-by-step algorithms, mathematical formulations, and trace tables for all 6 prescribed syllabus laboratory problems.

LAB PROBLEM 1: FIND THE LARGEST OF THREE NUMBERS (A, B, C)

Problem Statement: Read three distinct numerical values *A*, *B*, *C* from standard input and determine which number holds the maximum value.

Mathematical Formulation: $\text{Max}(A, B, C) = \max(A, \max(B, C))$.

```
Algorithm FindLargest(A, B, C):
1. START
2. INPUT A, B, C
3. IF (A >= B) THEN:
    IF (A >= C) THEN:
        DISPLAY "A is Largest"
    ELSE:
        DISPLAY "C is Largest"
    ELSE:
        IF (B >= C) THEN:
            DISPLAY "B is Largest"
        ELSE:
            DISPLAY "C is Largest"
4. STOP
```

Trace Table (*A* = 45, *B* = 78, *C* = 23):

Step	Condition Checked	Boolean Evaluation	Action Taken
1	Input <i>A</i> = 45, <i>B</i> = 78, <i>C</i> = 23	True	Variables assigned in memory
2	<i>A</i> ≥ <i>B</i> ⇒ 45 ≥ 78	False	Branch to Else (<i>B</i> > <i>A</i>)
3	<i>B</i> ≥ <i>C</i> ⇒ 78 ≥ 23	True	Execute: Display "B is Largest: 78"

LAB PROBLEM 2: CALCULATE SUM OF FIRST *N* NATURAL NUMBERS

Problem Statement: Compute the cumulative arithmetic sum $S = \sum_{i=1}^N i = 1 + 2 + 3 + \dots + N$ for a positive integer *N*.

Closed-Form Formula: $S = \frac{N(N+1)}{2}$. Time Complexity: Iterative $O(N)$, Closed-Form $O(1)$.

```
Algorithm SumOfNaturalNumbers(N):
1. START
2. INPUT N
3. INITIALIZE Sum = 0, Counter = 1
4. WHILE (Counter <= N) DO:
    Sum = Sum + Counter
    Counter = Counter + 1
5. DISPLAY "Sum is:", Sum
6. STOP
```

Trace Table for *N* = 5:

Iteration	Counter (<i>i</i>)	Condition (<i>i</i> ≤ 5)	Sum Expression	Updated Sum
Initial	1	-	0	0
1	1	1 ≤ 5 (True)	0 + 1	1
2	2	2 ≤ 5 (True)	1 + 2	3
3	3	3 ≤ 5 (True)	3 + 3	6
4	4	4 ≤ 5 (True)	6 + 4	10
5	5	5 ≤ 5 (True)	10 + 5	15
Exit	6	6 ≤ 5 (False)	Loop Terminates	Final: 15

LAB PROBLEM 3: COMPUTE FACTORIAL OF A NUMBER (N!)

Mathematical Formulation: $N! = \prod_{i=1}^N i = 1 \times 2 \times \cdots \times N$, with base case $0! = 1$.

```
Algorithm ComputeFactorial(N):
1. START
2. INPUT N
3. IF (N < 0) THEN:
    DISPLAY "Error: Negative numbers have no real factorial"
    GOTO Step 7
4. INITIALIZE Fact = 1, i = 1
5. WHILE (i <= N) DO:
    Fact = Fact * i
    i = i + 1
6. DISPLAY "Factorial is:", Fact
7. STOP
```

Trace Table for $N = 4$:

Step	Loop Variable (i)	Condition ($i \leq 4$)	Calculation ($Fact \times i$)	Updated Fact
Init	1	-	-	1
1	1	$1 \leq 4$ (True)	1×1	1
2	2	$2 \leq 4$ (True)	1×2	2
3	3	$3 \leq 4$ (True)	2×3	6
4	4	$4 \leq 4$ (True)	6×4	24
Exit	5	$5 \leq 4$ (False)	Loop Terminated	Final: 24

LAB PROBLEM 4: GENERATE FIBONACCI SEQUENCE UP TO N TERMS

Recurrence Relation: $F_0 = 0, F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.

```
Algorithm GenerateFibonacci(N):
1. START
2. INPUT N
3. IF (N <= 0) THEN GOTO Step 8
4. INITIALIZE T1 = 0, T2 = 1, Count = 1
5. WHILE (Count <= N) DO:
    DISPLAY T1
    NextTerm = T1 + T2
    T1 = T2
    T2 = NextTerm
    Count = Count + 1
6. STOP
```

Trace Table for $N = 6$:

Term Number	Displayed Value (T_1)	NextTerm ($T_1 + T_2$)	New T_1	New T_2
1	0	$0 + 1 = 1$	1	1
2	1	$1 + 1 = 2$	1	2
3	1	$1 + 2 = 3$	2	3
4	2	$2 + 3 = 5$	3	5
5	3	$3 + 5 = 8$	5	8
6	5	$5 + 8 = 13$	8	13

LAB PROBLEM 5: CHECK WHETHER A NUMBER IS PRIME

Mathematical Definition: An integer $N > 1$ is prime if and only if it has no positive divisors other than 1 and itself. Optimization: Test trial division up to $\lfloor \sqrt{N} \rfloor$, because if $N = a \times b$, at least one factor must satisfy $a \leq \sqrt{N}$.

```
Algorithm CheckPrime(N):
1. START
2. INPUT N
3. IF (N <= 1) THEN:
    DISPLAY "N is neither prime nor composite"
    GOTO Step 8
4. INITIALIZE IsPrime = True, Divisor = 2
5. WHILE (Divisor * Divisor <= N) DO:
    IF (N MOD Divisor == 0) THEN:
        IsPrime = False
        BREAK
    Divisor = Divisor + 1
6. IF (IsPrime == True) THEN:
    DISPLAY "N is a Prime Number"
ELSE:
    DISPLAY "N is a Composite Number"
7. STOP
```

Trace Table for Test Case $N = 29$ ($\lfloor \sqrt{29} \rfloor = 5$):

Step	Divisor	$\text{Divisor}^2 \leq 29$	$29 \bmod \text{Divisor}$	IsPrime State	Loop Action
Init	2	-	-	True	Enter Loop
1	2	$4 \leq 29$ (True)	$29 \bmod 2 = 1 \neq 0$	True	Divisor $\rightarrow$ 3
2	3	$9 \leq 29$ (True)	$29 \bmod 3 = 2 \neq 0$	True	Divisor $\rightarrow$ 4
3	4	$16 \leq 29$ (True)	$29 \bmod 4 = 1 \neq 0$	True	Divisor $\rightarrow$ 5
4	5	$25 \leq 29$ (True)	$29 \bmod 5 = 4 \neq 0$	True	Divisor $\rightarrow$ 6
Exit	6	$36 \leq 29$ (False)	-	True	Terminate: 29 is Prime

Trace Table for Test Case $N = 35$ ($\lfloor \sqrt{35} \rfloor = 5$):

Step	Divisor	$\text{Divisor}^2 \leq 35$	$35 \bmod \text{Divisor}$	IsPrime State	Loop Action
1	2	$4 \leq 35$ (True)	$35 \bmod 2 = 1 \neq 0$	True	Divisor $\rightarrow$ 3
2	3	$9 \leq 35$ (True)	$35 \bmod 3 = 2 \neq 0$	True	Divisor $\rightarrow$ 4
3	4	$16 \leq 35$ (True)	$35 \bmod 4 = 3 \neq 0$	True	Divisor $\rightarrow$ 5
4	5	$25 \leq 35$ (True)	$35 \bmod 5 = 0$	False	Break: 35 is Composite

LAB PROBLEM 6: LINEAR SEARCH IN AN ARRAY

Problem Statement: Search for a target key element K in an unsorted array A of size N .

Complexity: Best Case $O(1)$ (target at index 0), Worst Case $O(N)$ (target at last index or absent), Space Complexity $O(1)$.

```
Algorithm LinearSearch(A, N, Key):
1. START
2. INPUT Array A, Size N, Search Key K
3. INITIALIZE Index = 0, Found = False
4. WHILE (Index < N) DO:
    IF (A[Index] == Key) THEN:
        DISPLAY "Element found at index:", Index
        Found = True
        BREAK
    Index = Index + 1
5. IF (Found == False) THEN:
    DISPLAY "Element not present in array"
6. STOP
```

Trace Table ($A = [42, 17, 89, 53, 26]$, $N = 5$, $\text{Key} = 89$):

Iteration	Index (i)	Condition ($i < 5$)	Array Element ($A[i]$)	Comparison ($A[i] == 89$)	Action
1	0	$0 < 5$ (True)	42	$42 == 89$ (False)	$i \rightarrow 1$
2	1	$1 < 5$ (True)	17	$17 == 89$ (False)	$i \rightarrow 2$
3	2	$2 < 5$ (True)	89	$89 == 89$ (True)	Found at Index 2; Break

Trace Table for Absent Key ($A = [42, 17, 89, 53, 26]$, $N = 5$, $\text{Key} = 99$):

Iteration	Index (i)	Condition ($i < 5$)	Array Element ($A[i]$)	Comparison ($A[i] == 99$)	Action
1	0	$0 < 5$ (True)	42	$42 == 99$ (False)	$i \rightarrow 1$
2	1	$1 < 5$ (True)	17	$17 == 99$ (False)	$i \rightarrow 2$
3	2	$2 < 5$ (True)	89	$89 == 99$ (False)	$i \rightarrow 3$
4	3	$3 < 5$ (True)	53	$53 == 99$ (False)	$i \rightarrow 4$
5	4	$4 < 5$ (True)	26	$26 == 99$ (False)	$i \rightarrow 5$
Exit	5	$5 < 5$ (False)	-	-	Display "Not present"

Academic References & Recommended Bibliography

1. Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organization and Design: The Hardware/Software Interface* (6th ed.). Morgan Kaufmann.
2. Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
3. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons.
4. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
5. Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
6. Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, 49(3), 33-35.
7. Mano, M. M. (2017). *Digital Logic and Computer Design*. Pearson Education India.
8. Stallings, W. (2016). *Computer Organization and Architecture: Designing for Performance* (10th ed.). Pearson.
9. IEEE / ACM Joint Task Force on Computing Curricula. (2020). *Computing Curricula 2020: Paradigms for Global Computing Education*. ACM/IEEE Computer Society.
10. ISO/IEC 5807:1985. *Information processing — Documentation symbols and conventions for data, program and system flowcharts, network charts and system resources charts*. International Organization for Standardization.